



**Universidad
Tecnológica
del Perú**

PROYECTO: ARQUITECTURA ORIENTADA AL SERVICIO

AVANCE N.º 1

Impacto de la Arquitectura Orientada a Servicios (SOA) en la eficiencia operativa del sistema MercadoLink en mercados minoristas en San Martín de Porres, Lima – 2026

Integrantes:

Victor Centeno Vasquez	U21215694
Jesus Jimenev Fernandez	U21221039
James Munayco Meneses	U21302439
Ivan Leonardo Hinostroza Anardo	U20307868

Docente: Ronald Angel Ayra Pinto

Lima-Perú

ÍNDICE

1. Introducción
2. Descripción del sistema de servicio web a desarrollar
 - 2.1 Problemática
 - 2.2 Solución propuesta
 - 2.3 Lenguajes y tecnologías a utilizar
3. Partes del servicio web
 - 3.1 Contrato
 - 3.2 Interfaz
 - 3.3 Implementación
 - 3.4 Lógica de negocio
 - 3.5 Datos
4. Tipos de servicios implementados
 - 4.1 Servicio de utilidad
 - 4.2 Servicio de entidad
 - 4.3 Servicio de tarea
5. Importancia de la seguridad en el servicio web
 - 5.1 Autenticación
 - 5.2 Autorización
 - 5.3 Confidencialidad
 - 5.4 Disponibilidad

6. Arquitectura de software en tiempos de ejecución
 - 6.1. Arquitectura rígida
 - 6.2. Arquitectura flexible
 - 6.3. Arquitectura adaptativa
7. Modelos de distribución y Licenciamiento
8. Diagramas técnicos
 - 8.1. Diagramas de interacción
 - 8.2. Diagramas de despliegue
 - 8.3. Diagramas de componentes
9. Desarrollo del servicio web
 - 9.1. Evidencia de código y estructura
 - 9.2. Pruebas de funcionamiento
 - 9.3. Descripción del avance
10. Conclusiones
11. Bibliografía

PROYECTO: ARQUITECTURA ORIENTADA AL SERVICIO

AVANCE N.º 1 – DISEÑO INICIAL DEL SERVICIO WEB

1. INTRODUCCIÓN

La Arquitectura Orientada al Servicio (SOA, por sus siglas en inglés Service-Oriented Architecture) es un paradigma de diseño de software que organiza las funcionalidades de un sistema en un conjunto de servicios independientes, reutilizables e interoperables que se comunican entre sí mediante interfaces estandarizadas. Según Erl (2016), SOA permite construir sistemas empresariales flexibles donde cada componente puede evolucionar, escalar o reemplazarse sin interrumpir el resto de la plataforma. Esta característica la convierte en uno de los enfoques más relevantes para el desarrollo de sistemas modernos distribuidos, especialmente en contextos donde múltiples actores, tecnologías y fuentes de datos deben integrarse de forma eficiente.

En el contexto del comercio minorista peruano, la digitalización de los mercados populares representa una necesidad urgente. La Asociación de Comerciantes del Mercado Popular de la urbanización Ingeniería, ubicada en el distrito de San Martín de Porres (SMP), Lima, enfrenta hoy en día una serie de ineficiencias operativas derivadas de la ausencia de herramientas tecnológicas adecuadas. El presente avance describe el diseño inicial de MercadoLink, un sistema de servicio web basado en SOA orientado a transformar la operación diaria de este mercado. Se exponen la problemática detectada, la solución propuesta, las tecnologías seleccionadas, las partes y tipos de servicios considerados, así como los mecanismos de seguridad que garantizarán la integridad y disponibilidad del sistema.

2. DEFINICIÓN DEL SISTEMA DE SERVICIO WEB

2.1 Sistema de servicio web a desarrollar

El sistema está dirigido a vendedores de puestos, proveedores y clientes mayoristas que operan en mercados populares peruanos, facilitando transacciones comerciales B2B de manera automatizada y trazable. Se llama en este caso: Sistema de Automatización de Pedidos B2B Aspropa basado en SOA. Está diseñado para organizaciones como mercados mayoristas, administradores de mercados populares (ej. Aspropa) y redes de proveedores, donde múltiples actores necesitan coordinar inventarios, pedidos, pagos y logística de forma interoperable sin depender de procesos manuales o sistemas legados fragmentados.

a. Dirigido a: Vendedores de puestos (dueños/administradores), proveedores de mercancía, clientes mayoristas (mercaderes) y administradores del mercado. Cada actor accede a funcionalidades específicas según su rol RBAC: vendedores gestionan inventario y confirman pedidos; proveedores publican catálogos y reciben órdenes; mayoristas crean pedidos multipuesto y pagan; administradores supervisan y resuelven disputas.

b. Organizaciones usuarias: Mercados populares organizados (ej. Aspropa), administradores de mercados mayoristas, redes de proveedores B2B y plataformas de

comercio informal digitalizado que requieren interoperabilidad entre actores con distintos niveles de madurez tecnológica (desde apps móviles hasta sistemas legacy).

Información manejada:

- Datos operativos: Inventarios por puesto/producto (stock, precios B2B, categorías), pedidos B2B.
- Datos de actores: Perfiles (RUC/DNI, roles, puestos asignados), historial de transacciones, permisos RBAC.
- Datos analíticos: Reportes de ventas (por vendedor/período), auditorías (con correlationId, timestamps, IP origen), métricas de stock y conciliaciones.

2.2 Problemática

Actualmente, la gestión del inventario se realiza de manera descentralizada mediante registros físicos en libretas o hojas de cálculo locales en cada puesto, sin mecanismos de sincronización en tiempo real con los proveedores. Los clientes mayoristas deben recorrer físicamente múltiples puestos durante varias decenas de minutos u horas diarias únicamente para verificar verbalmente la disponibilidad de productos, anotando manualmente los datos en dispositivos móviles o papel.

La coordinación de pedidos se efectúa principalmente a través de aplicaciones de mensajería instantánea como WhatsApp, complementada con transferencias bancarias fragmentadas vía Yape y Plin. Esta práctica genera un volumen descontrolado de mensajes de texto, notas de voz y chats grupales sin orden de prioridad ni trazabilidad formal. Los proveedores reciben decenas de solicitudes diarias desorganizadas, mientras los vendedores dedican entre 2 y 3 horas diarias exclusivamente a conciliar múltiples transferencias electrónicas sin correlación centralizada con los pedidos correspondientes.

- El flujo típico de un pedido comercial involucra anotaciones manuales, envío de fotografías por WhatsApp, confirmaciones verbales, despacho sin garantías de disponibilidad y pagos distribuidos entre múltiples cuentas. Esta cadena carece de confirmación atómica que vincule simultáneamente stock disponible, aprobación de pago y compromiso de despacho, resultando en discrepancias sistemáticas del 15-25% en cantidades y precios reportados.
- Finalmente, la ausencia total de sistemas de auditoría unificados impide establecer evidencia objetiva sobre quién solicitó qué producto, cuánto se pagó efectivamente y cuándo se concretó la entrega, obligando a resolver disputas comerciales mediante confrontaciones basadas en memoria o conversaciones fragmentadas.

Existen problemas observados en el mercado popular que pueden solucionarse con tal transformación y bonus:

1. Ineficiencia operativa genera pérdida económica directa: Si un

mayorista pierde 3 horas/día coordinando 10 pedidos manualmente, equivale a S/30-50 diarios solo en tiempo. Multiplicado por 100 mayoristas = S/3,000-5,000/día perdidos en eficiencia.

2. Errores humanos sistemáticos: Sin validación atómica, el 20% de pedidos tienen discrepancias (cantidad, precio, stock). Un error de S/100 x 50 pedidos/semana = S/5,000/semana en ajustes. Proveedores rechazan pedidos por falta de claridad = oportunidad perdida.

3. Escalabilidad bloqueada: Con 50 puestos, el caos es manejable con WhatsApp. Con 500 puestos, colapsa. No hay forma de crecer sin centralización. Mayoristas abandonan el mercado por lentitud.

4. Riesgo financiero sin trazabilidad: Pagos Yape/Plin sin conciliación = vendedor recibe S/1,000 pero entrega mercancía por S/1,200 (sin ajuste). Pérdida neta 10-20% sin evidencia para reclamar.

5. Proveedores desorganizados pierden mercado: Sin pedidos consolidados, proveedores pequeños no pueden competir con supermercados digitalizados. Mercado informal pierde competitividad frente a Plaza Vea/Makro.

Estas prácticas generan tiempos muertos operativos significativos: los mayoristas pierden entre una buena parte de su jornada laboral en coordinación logística no productiva; los errores humanos sistemáticos afectan al 20% de las operaciones comerciales; y los vendedores destinan hasta 3 horas diarias a conciliaciones manuales de pagos. En términos monetarios, considerando 100 mayoristas operando diariamente, se estima una pérdida agregada de S/3,000-5,000 por ineficiencias temporales y adicionales de S/5,000 semanales por discrepancias comerciales. Según nuestro equipo se ha observado que se mueven miles de soles en una semana y llegan mucho más al año, contablemente causan impactos la falta de medidas así como que nuestra opción justifica tanto reducción de costes así como mejor canalización e inventariado ordenando lo que es en sus procesos de negocio en general.

2.3 Solución propuesta

La problemática identificada en el mercado popular puede ser atendida mediante la implementación de un sistema basado en servicios web, orientado a automatizar los procesos críticos de abastecimiento, reposición, gestión de pedidos, evaluación de proveedores y control de recepción de mercadería. La finalidad de esta solución no consiste únicamente en digitalizar actividades aisladas, sino en integrar operaciones que actualmente se desarrollan de forma manual, dispersa y con escasa trazabilidad, generando demoras, errores de coordinación y pérdida de control sobre la información operativa.

En este contexto, la automatización se produce principalmente en los puntos del proceso donde hoy existen validaciones manuales, comunicación informal entre actores y registros no sincronizados. El sistema propuesto reemplaza dichas prácticas por flujos estructurados que permiten registrar solicitudes, validar condiciones de negocio, actualizar estados y notificar resultados de manera inmediata, utilizando servicios

interoperables que comparten información en tiempo real entre vendedores, encargados del mercado, almacén y proveedores.

En el proceso de abastecimiento o compras B2B, la automatización se aplica desde el momento en que el vendedor o punto de venta detecta una necesidad de reposición. En lugar de depender de avisos informales o solicitudes transmitidas de manera manual, el sistema permite registrar digitalmente la solicitud de abastecimiento y canalizar hacia el área de compras. A partir de ello, se automatizan tareas como la validación de políticas internas, la generación de la orden de compra, el envío formal al proveedor, el registro de confirmaciones de disponibilidad y el seguimiento del estado de atención. De esta manera, se reduce la dependencia de llamadas, mensajes desordenados o coordinaciones informales que suelen retrasar la reposición efectiva de productos.

En el proceso de reposición de stock, el sistema automatiza la detección de productos por debajo del nivel mínimo, el registro de solicitudes de reposición y la actualización del inventario una vez recibida la mercadería. Esto significa que la información deja de mantenerse en libretas, apuntes aislados o reportes verbales, y pasa a gestionarse mediante servicios que reflejan el estado real del stock de manera inmediata. La automatización en esta etapa resulta clave porque evita tiempos muertos derivados de la falta de visibilidad sobre la existencia de productos, mejora la planificación del abastecimiento y permite que tanto almacén como vendedores operen con información consistente.

En la gestión de pedidos, el servicio web automatiza el registro del requerimiento, la revisión de disponibilidad interna, la consolidación del pedido externo cuando corresponda, la comunicación con el proveedor y el control de recepción de mercadería. Este punto es especialmente importante porque uno de los principales problemas del mercado popular radica en el desorden comunicacional entre quienes solicitan productos, quienes gestionan la compra y quienes finalmente abastecen. Al estructurar este flujo dentro de un sistema, cada pedido queda formalizado, con datos verificables sobre cantidades, condiciones, fecha de atención y resultado final. Esto disminuye errores de interpretación, reduce duplicidad de solicitudes y facilita la continuidad del proceso sin depender exclusivamente de la memoria o de conversaciones informales.

En relación con la gestión y evaluación de proveedores, la automatización se concreta en el registro digital de criterios de evaluación, la comparación de propuestas, la aprobación o descarte de proveedores y el seguimiento de su desempeño a lo largo del tiempo. Este aspecto resulta relevante porque, en contextos donde la selección de proveedores suele realizarse sobre la base de experiencias informales o decisiones poco documentadas, el sistema permite construir un historial verificable de cumplimiento, calidad y tiempos de entrega. Como consecuencia, la organización puede tomar decisiones de compra con mayor sustento, reducir la improvisación y establecer relaciones comerciales más ordenadas y controlables.

Asimismo, en el proceso de logística y cumplimiento de entrega, el servicio web automatiza el registro de recepción de mercadería, la verificación de cantidades y condiciones, la aceptación o el reporte de diferencias, así como la confirmación de

recepción por parte del vendedor del puesto. Aquí la automatización se concentra en una fase especialmente sensible, ya que es donde suelen producirse observaciones sobre faltantes, daños, diferencias entre lo solicitado y lo recibido, o inconsistencias entre áreas. Al registrar cada evento del proceso en el sistema, se mejora la trazabilidad operativa y se facilita la identificación del punto exacto en el que se generó una incidencia.

Los beneficios de esta automatización son concretos. En primer lugar, se reduce el tiempo invertido en coordinación manual entre actores, ya que la información se canaliza a través de servicios estructurados y no mediante múltiples intercambios informales. En segundo lugar, se mejora la calidad de la información, al evitar duplicidad de registros, omisiones y diferencias entre lo comunicado y lo ejecutado. En tercer lugar, se fortalece la integración entre las áreas involucradas, porque todos los actores consultan y actualizan una misma base operativa. Finalmente, se incrementa la seguridad y trazabilidad del proceso, ya que cada solicitud, validación, recepción o incidencia queda registrada formalmente dentro del sistema, permitiendo un mejor control y facilitando la toma de decisiones.

2.4 Lenguajes y tecnologías a utilizar

Para el desarrollo del sistema propuesto se emplea un conjunto de tecnologías orientadas a garantizar interoperabilidad, mantenibilidad, escalabilidad y facilidad de integración dentro de una arquitectura basada en servicios web. La selección realizada responde tanto a los requerimientos funcionales del sistema como a la necesidad de contar con herramientas ampliamente utilizadas en entornos empresariales y académicos.

El lenguaje de programación principal será Java, debido a su solidez en el desarrollo de aplicaciones empresariales, su madurez tecnológica y su amplia compatibilidad con frameworks orientados a la construcción de servicios web. Java ofrece una estructura robusta, tipado fuerte y una gran capacidad para desarrollar soluciones mantenibles, escalables y seguras, lo cual resulta apropiado para un sistema que maneja procesos críticos relacionados con pedidos, inventario, proveedores y control de operaciones. El tipo de servicio web que se utilizará será REST, ya que permite construir interfaces ligeras, ampliamente compatibles y fáciles de consumir desde aplicaciones web y móviles. REST resulta conveniente para este proyecto porque simplifica la comunicación entre cliente y servidor mediante el uso de métodos HTTP estandarizados, favorece la interoperabilidad entre componentes y se adapta mejor a un entorno donde se requiere intercambio frecuente de información operativa en tiempo real.

Como framework principal de desarrollo backend se utilizará Spring Boot, debido a que facilita la creación de servicios REST de manera rápida y estructurada, reduce la configuración manual y proporciona soporte integrado para seguridad, acceso a datos, validación, control de dependencias y escalabilidad. Su uso es especialmente adecuado en este proyecto porque permite organizar la lógica del sistema en servicios bien definidos, alineados con el enfoque de arquitectura orientada a servicios. En cuanto a la base de datos, se emplea SQL mediante PostgreSQL, por tratarse de un sistema gestor de base de datos relacional confiable, estable y adecuado para el manejo de

información estructurada. PostgreSQL permitirá almacenar de manera consistente datos vinculados con usuarios, inventarios, pedidos, proveedores, recepciones, evaluaciones y reportes, manteniendo integridad referencial, control transaccional y facilidad para consultas complejas.

Respecto a los frameworks y tecnologías complementarias, se considera el uso de TypeScript para el desarrollo del frontend, ya que permite construir interfaces más seguras y mantenibles mediante tipado estático. Asimismo, se contempla el uso de Power BI para la visualización de indicadores y reportes analíticos, D3.js para gráficos interactivos, Git y GitHub para el control de versiones y gestión del desarrollo colaborativo, e Izipay como plataforma de integración para el procesamiento de pagos dentro del sistema. Estas herramientas complementan la solución al cubrir necesidades de presentación de datos, seguimiento del desarrollo, análisis y transacciones económicas. El formato de intercambio de datos será principalmente JSON, debido a su ligereza, claridad estructural y facilidad de integración con servicios REST. JSON permite representar información de manera sencilla y comprensible tanto para el backend como para el frontend, lo cual facilita el intercambio de datos entre módulos del sistema y mejora la interoperabilidad con servicios externos.

En conjunto, estas tecnologías han sido elegidas porque ofrecen un equilibrio adecuado entre rendimiento, facilidad de implementación, compatibilidad con arquitecturas modernas y capacidad de evolución futura. Su adopción permitirá desarrollar una solución coherente con los objetivos del proyecto, asegurando una base técnica sólida para automatizar los procesos críticos del mercado popular de manera ordenada y sostenible.

3. PARTES DEL SERVICIO WEB

Según Mohor (2006) El contrato de servicio proporciona una especificación informal del propósito, funcionalidad, restricciones, y del uso del servicio. El formato de esta especificación puede variar, dependiendo del tipo de servicio. Puede contener una definición de interfaz formal basada en lenguajes como IDL o WSDL.

3.1 Contrato

3.1.1. Propósito y alcance del servicio

El concepto de negocio encapsulado en SOA se refiere a la definición de un servicio que agrupa una funcionalidad empresarial específico, en el caso descrito de la aplicación en la compañía Aspropa, ello está dentro del rubro de automatización de datos, con un alto nivel del independientemente de su implementación técnica e idempotencia aplicable debido a la implementación del servicio B2B debido a la presencia de proveedores, vendedores, clientes y mercaderes, se diferenciará a un contrato que especifica propósito, interfaz pública y reglas de uso en continuación.

En un mercado popular, el servicio “gestión de pedidos B2B” encapsula la lógica de negocio para verificar disponibilidad de stock en los puestos, procesar órdenes entre vendedores y proveedores, actualizar inventarios en tiempo real y confirmar transacciones, permitiendo que mercaderes externos o clientes mayoristas lo consuman a través de interfaces estandarizadas con APIs de Yape y Plin vía Izipay o las primeras

mismas, sin necesidad de conocer los detalles internos de cada puesto ni de los sistemas legados de los participantes con tantas restricciones al momento de realizar pagos.

3.1.2. Requisitos funcionales por actores

Actor	Descripción General del Actor	Requerimientos Funcionales Principales
Vendedor de Puesto (dueño o administrador del puesto en el mercado)	Usuario interno que gestiona su inventario, recibe pedidos B2B y actualiza stock en tiempo real desde su puesto.	- Autenticarse en el sistema y gestionar su perfil de puesto. - Consultar y actualizar inventario propio (agregar, modificar, eliminar productos). - Recibir y confirmar pedidos B2B entrantes. - Notificar disponibilidad de stock a proveedores y clientes. - Generar reportes de ventas y stock por día/semana. - Acceder al servicio “Actualización de Inventario” de forma idempotente.
Proveedor	Entidad externa que suministra mercancía a uno o varios puestos del mercado.	- Registrarse y publicar catálogo de productos con precios B2B. - Recibir órdenes de compra automáticas desde los puestos. - Consultar estado de pedidos y confirmar envíos. - Actualizar precios y disponibilidad en tiempo real vía servicio SOA. - Recibir notificaciones de pago y liquidación.
Cliente Mayorista y Mercader	Comprador externo que realiza pedidos grandes a múltiples puestos del mercado.	- Buscar y consultar disponibilidad de productos en diferentes puestos. - Crear y enviar pedidos B2B (carrito multi-puesto). - Ver historial de pedidos, facturas y estado de entrega. - Recibir confirmaciones y actualizaciones automáticas vía interfaz estandarizada (API). - Realizar pagos y descargas de comprobantes.
Administrador del Mercado (Aspropa)	Usuario central que supervisa todo el ecosistema, gestiona usuarios y monitorea transacciones.	- Gestionar registros de puestos, proveedores y usuarios. - Monitorear transacciones B2B y reportes globales (ventas totales, stock agregado). - Configurar reglas de negocio y contratos de servicios SOA. - Resolver disputas y aprobar/rechazar operaciones. - Generar auditorías y reportes analíticos del sistema.

3.1.3. Requisitos no funcionales por actores

Esta tabla cubre las características de calidad del sistema en su conjunto, con énfasis en aspectos clave de SOA (reutilización, independencia técnica, interoperabilidad, idempotencia, etc.).

Categoría	Requerimiento Funcional	No Detalle / Métrica Sugerida
Rendimiento	El sistema debe responder a consultas y transacciones en tiempo razonable, incluso bajo carga.	Tiempo de respuesta promedio < 2 segundos para consultas de stock y < 5 segundos para procesamiento de pedidos B2B. Soporte mínimo de 200 transacciones concurrentes en horas pico.
Escalabilidad	Capacidad para crecer con el número de puestos, proveedores y volumen de transacciones.	Soporte horizontal (agregar más puestos o servicios sin impacto significativo). Diseño para escalar a 50 a más puestos y miles de transacciones diarias.
Disponibilidad	El sistema debe estar accesible la mayor parte del tiempo.	Disponibilidad $\geq 99\%$ (downtime máximo de ~3.6 horas al mes). Soporte a operaciones 24/7 con mantenimiento programado.
Seguridad	Protección de datos sensibles y control de accesos.	Autenticación fuerte (OAuth 2.0 o JWT), cifrado de datos en tránsito y reposo (HTTPS/TLS), roles y permisos RBAC, cumplimiento de normativas de protección de datos (ej. Ley Federal de Protección de Datos en México).
Idempotencia y Confiabilidad	Las operaciones críticas (ej. creación de pedidos) deben ser independientes.	Un mismo pedido enviado varias veces por fallo de red debe procesarse sólo una vez. Uso de identificadores únicos y mecanismos de reintento seguros.
Interoperabilidad	Facilidad de integración con sistemas existentes y externos.	Servicios expuestos mediante contratos estandarizados WSDL y SOAP. Soporte a sistemas legados de puestos y ERP de proveedores.
Usabilidad	Interfaz intuitiva para usuarios con diverso nivel técnico (vendedores de mercado).	Interfaz web y móvil responsive. Tiempo de aprendizaje mínimo. Soporte multilinguaje (español principal).
Mantenibilidad y Reusabilidad	Los servicios SOA deben ser modulares e independientes.	Cada servicio encapsula una funcionalidad de negocio clara, con bajo acoplamiento y alta cohesión. Facilidad para agregar nuevos servicios sin afectar los existentes.
Auditoría y Trazabilidad	Registro completo de todas las transacciones B2B.	Logs detallados con timestamps, usuario y servicio invocado. Posibilidad de auditoría completa para disputas o reportes fiscales.
Portabilidad y Compatibilidad	Funcionamiento en diferentes entornos y dispositivos.	Compatible con navegadores modernos, dispositivos móviles y entornos cloud/on-premise.

3.1.4. Política y Reglas

Las políticas y reglas de negocio definen el comportamiento esperado de los servicios SOA encapsulados. Cada servicio opera bajo un contrato formal que especifica su propósito, interfaz pública, prerequisites y reglas de uso, garantizando independencia técnica, reutilización y consistencia. Estas reglas se implementan como políticas adjuntas al contrato de cada servicio, permitiendo que el sistema sea gobernado de forma centralizada mientras los servicios permanecen apegados a este mismo.

Política Regla	Descripción	Aplicación en Servicios SOA	Actor(es) Afectados
Idempotencia en Operaciones Críticas	Una misma solicitud enviada múltiples veces (por fallos de red o reintentos) debe producir el mismo efecto que si se enviara una sola vez.	El servicio “ProcesarPedidoB2B” utilizará identificadores únicos de transacción (UUID) para evitar duplicados automáticos.	Vendedores, Proveedores, Clientes Mayoristas
Reglas de Validación de Pedidos	Los pedidos B2B solo se procesan si existe stock suficiente en el puesto, el proveedor está activo y se cumplen las condiciones de pago mínimas.	El servicio verifica disponibilidad en tiempo real antes de confirmar. Rechazo automático si no se cumplen reglas.	Todos los actores
Actualización en Tiempo Real de Inventario	Cualquier cambio en stock (venta, recepción de mercancía) debe reflejarse inmediatamente en todos los servicios que consulten inventario.	El servicio “ActualizarInventario” es atómico y notifica a servicios suscritos (patrón publish-subscribe).	Vendedores de Puesto, Proveedores
Políticas de Precios y Descuentos	Precios B2B personalizados por cliente, volumen o acuerdo comercial. Descuentos aplicables según reglas predefinidas (ej. por cantidad o fidelidad).	El servicio “CotizaciónB2B” aplica reglas dinámicas según perfil del cliente.	Proveedores, Clientes Mayoristas
Reglas de Cancelación y Devoluciones	Cancelación permitida solo antes de confirmar el envío. Devoluciones sujetas a inspección y aprobación del vendedor/proveedor.	Servicio “GestionarPedido” registra estado y aplica reglas según etapa del pedido.	Todos los actores
Límites de Uso y Cuotas	Límite diario/semanal de pedidos o consultas por usuario para prevenir abusos.	Gobernanza SOA aplica throttling en el nivel de servicio (ej. 100 consultas/hora por puesto).	Administrador del Mercado, Vendedores
Auditoría y Trazabilidad	Toda transacción debe registrar quién, qué, cuándo y desde qué servicio se ejecutó.	Todos los servicios registran logs con trazabilidad completa para auditorías fiscales o resolución de disputas.	Administrador del Mercado

3.1.5. Condiciones de uso

Las condiciones de uso respectivas están referentes al uso sugerible y reglamentado de todo el sistema, tanto para cada actor representado como para el usuario futurible. Estas son las dichas condiciones:

A. Objeto: Las presentes Condiciones regulan el acceso y uso de la plataforma digital Aspropa, que proporciona servicios SOA para la automatización de procesos B2B entre puestos del mercado, proveedores, vendedores y clientes mayoristas (mercaderes). El uso de la plataforma implica la aceptación expresa y sin reservas de estas condiciones, así como de las políticas de negocio y privacidad vigentes.

B. Usuarios y Registro: Sólo pueden registrarse personas morales o físicas con actividad comercial (mayores de edad o empresas legalmente constituidas). Cada usuario (Vendedor de Puesto, Proveedor, Cliente Mayorista o Administrador) recibirá credenciales de acceso únicas. El usuario es responsable de la confidencialidad de sus credenciales. Cualquier operación realizada con ellas se considerará efectuada por el titular.

C. Servicios Ofrecidos: La plataforma expone servicios reutilizables bajo arquitectura SOA, tales como:

- Consulta y actualización de inventario
- Procesamiento de pedidos B2B
- Gestión de cotizaciones y pagos
- Reportes y auditorías
- Cada servicio se rige por su contrato específico, que define interfaz, reglas de uso, prerequisites y niveles de servicio (SLA).

D. Reglas de Uso Aceptable: Queda prohibido: usar la plataforma para fines ilícitos, cargar información falsa, realizar scraping masivo, intentar sobrecargar los servicios o eludir mecanismos de seguridad. Los usuarios deben proporcionar información veraz y mantenerla actualizada. Está prohibido compartir credenciales o permitir el acceso a terceros no autorizados. Las operaciones deben cumplir con la legislación comercial mexicana y las normas fiscales aplicables.

E. Responsabilidades

- Aspropa (Administrador) se compromete a mantener la disponibilidad del sistema según los niveles definidos en los requisitos no funcionales ($\geq 99\%$).
- Los usuarios son responsables de la exactitud de los datos que ingresan y de cumplir las reglas de negocio.

- Asproa no se hace responsable por pérdidas derivadas de errores del usuario, fallos en sistemas externos o fuerza mayor.

F. Idempotencia y Confiabilidad: Todas las operaciones críticas (especialmente creación y modificación de pedidos) son idempotentes. El reenvío accidental de una solicitud no generará duplicados ni cargos adicionales.

G. Pagos, Comisiones y Facturación: La plataforma podrá aplicar comisiones por transacción o suscripción según tarifa vigente (publicada en el portal). Los pagos se procesarán mediante los medios integrados y estarán sujetos a las condiciones del proveedor de pagos.

H. Protección de Datos Personales: El tratamiento de datos se realiza conforme a la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP). Los usuarios pueden ejercer sus derechos ARCO mediante solicitud al administrador. Se aplican medidas de seguridad técnicas y organizativas adecuadas.

I. Modificaciones: Asproa se reserva el derecho de modificar estas condiciones, notificando con al menos 15 días de antelación. El uso continuado tras la modificación implica aceptación.

J. Terminación y Suspensión: Asproa podrá suspender o cancelar el acceso por incumplimiento grave de estas condiciones, sin perjuicio de las acciones legales correspondientes.

K. Ley Aplicable y Jurisdicción: Estas condiciones se rigen por la legislación mexicana. Cualquier controversia se someterá a los tribunales competentes de las prefecturas de Lima.

L. Contacto: Para dudas o notificaciones: Vía correo del administrador de Asproa o a través del módulo de soporte de la plataforma. Esta sección refuerza el enfoque SOA al vincular las reglas de negocio directamente con los contratos de servicio y resalta aspectos como idempotencia, interoperabilidad y gobernanza.

3.2 Interfaz

Se define la capa de contrato público de un servicio en SOA, especificando operaciones, entradas/salidas y cómo interactúan los clientes. Incluye descripción técnica, formatos de mensajes y protocolos/bindings para interoperabilidad. Service Interface define la capa de contrato público de un servicio en SOA, especificando operaciones, entradas y salidas y cómo interactúan los clientes. Incluye descripción técnica, formatos de mensajes y protocolos/bindings para interoperabilidad. Documenta operaciones (request-response, one-way), tipos de datos (XML Schema) y endpoints. Usa WSDL y/o OpenAPI para especificar abstractamente: port Types (operaciones), messages (estructuras) y versiones para compatibilidad.

3.2.1 Protocolos y bindings

Los servicios del sistema se expondrán mediante el protocolo HTTP/HTTPS bajo un estilo arquitectónico RESTful, utilizando métodos estándar como GET, POST, PUT, PATCH y DELETE. La mayor parte de las operaciones seguirá un patrón de interacción request-response, mientras que ciertos eventos externos, como las notificaciones de pago enviadas por Izipay, se manejan mediante interacciones asíncronas de tipo one-way a través de Webhook o IPN. El binding tecnológico se realizará sobre REST/HTTP con intercambio de mensajes en formato JSON. La especificación abstracta del contrato de servicio se documentará mediante WSDL, que permitirá definir operaciones equivalentes a tipo de puertos, estructuras de mensajes equivalentes a messages, esquemas de datos, códigos de respuesta y políticas de versionado, especialmente mediante endpoints versionados del tipo numérico.

Elemento	Aplicación en tu sistema
Protocolo	HTTP/HTTPS
Estilo de servicio	REST
Binding	REST sobre HTTP con JSON
Patrón principal	Request-response
Patrón complementario	One-way asíncrono para IPN/Webhook de Izipay
Contrato técnico	WSDL
Esquemas de datos	JSON Schema; académicamente comparable a XML Schema
Versionado	/api/v1/..., /api/v2/...

3.2.2 Documentación

Se presentará en WSDL como la estructura de proyecto:

Primera versión	Segunda versión
/servicios/ /gestion-pedidos/ /contrato/ gestion-pedidos-v1.0.md gestion-pedidos-v2.0.md /interfaz/ pedidos-v1.0.wsdl pedidos-v2.0.wsdl /xsd/ pedidos-common-v1.0.xsd pedidos-messages-v1.0.xsd pedidos-types-v1.0.xsd pedidos-common-v2.0.xsd pedidos-messages-v2.0.xsd pedidos-types-v2.0.xsd /samples/	/servicios/ README.md aspropa-database-v1.0.sql /interfaz/ Gestión-Pedidos-v1.0.wsdl Actualización-Inventario-v1.0.wsdl Gestión-Proveedores-v1.0.wsdl LogísticaEntrega-v1.0.wsdl /xsd/ aspropa-common-types-v1.0.xsd pedidos-types-v1.0.xsd pedidos-messages-v1.0.xsd inventario-types-v1.0.xsd inventario-messages-v1.0.xsd proveedores-types-v1.0.xsd proveedores-messages-v1.0.xsd

crear-pedido-request-v1.xml crear-pedido-response-v1.xml notificar-envio-request-v1.xml /policies/ security-policy-v1.xml /docs/ matriz-versiones.md codigos-error.md reglas-validacion.md	logistica-types-v1.0.xsd logistica-messages-v1.0.xsd
--	---

3.2.3 Manual de excepciones

Implementar en lo que son los impactos de la naturaleza de lo que pueda aplicarse o afectarse en la página web, tales situaciones inusuales

Código	Excepción	Aplica a	Sitio donde ocurre	Tratamiento
EX-AUTH-001	Credenciales inválidas	Todos los actores	Login con usuario/clave/token incorrecto.	Responder 401 con mensaje seguro, registrar intento y permitir nuevo intento controlado.
EX-AUTH-002	Acceso no autorizado	Todos	El actor intenta operar fuera de su rol o alcance.	Responder 403 y auditar el evento.
EX-INV-001	Producto no encontrado	Vendedor, Proveedor, Cliente	Se consulta o actualiza un producto inexistente.	Responder 404 con identificador de recurso y trazabilidad.
EX-INV-002	Stock insuficiente	Vendedor, Cliente Mayorista	El pedido excede el stock disponible.	Rechazar operación, sugerir stock parcial o backorder si la regla de negocio lo permite.
EX-INV-003	Conflicto de actualización de inventario	Vendedor, Proveedor	Dos actualizaciones simultáneas generan inconsistencia.	Responder 409, exigir versión/revisión o relectura del recurso.
EX-INV-004	Operación idempotente duplicada	Vendedor, Proveedor	Se repite una solicitud con la misma Idempotency-Key.	No duplicar efecto, devolver el resultado previo o estado actual.
EX-INV-005	Idempotency-Key y inválida	Vendedor, Proveedor	La clave no cumple formato o cambia el cuerpo de la solicitud. github	Responder 400 y no procesar la mutación.

EX-ORD-001	Pedido inválido	Cliente, Vendedor	Faltan campos, cantidades o reglas mínimas de compra. swagger	Responder 400 con detalle de campos inválidos.
EX-ORD-002	Pedido ya confirmado/cerrado	Vendedor, Cliente	Se intenta editar o cancelar un pedido cerrado.	Responder 409 por conflicto de estado. +1
EX-ORD-003	Multi-puesto inconsistente	Cliente Mayorista	El carrito combina datos incompatibles entre puestos.	Rechazar parcialmente o dividir en órdenes separadas según política.
EX-PROV-001	Catálogo no disponible	Proveedor, Vendedor	El catálogo externo no responde o está desactualizado.	Marcar datos como no confiables, activar reconciliación y aviso.
EX-PRICE-001	Precio desactualizado	Proveedor, Cliente, Vendedor	El precio cambió antes de confirmar la orden.	Revalidar antes del cierre y pedir confirmación.
EX-PAY-001	Pago rechazado	Cliente, Administrador	El procesador rechaza el cobro. catalogmachine	Mantener pedido pendiente o anulado según regla, notificar sin exponer detalle sensible.
EX-PAY-002	Comprobante no generado	Cliente, Administrador	El pago existe pero falla la emisión del comprobante.	Dejar transacción en estado conciliación pendiente y reintentar emisión.
EX-NOTIF-001	Notificación no entregada	Todos	Falla correo, webhook o mensaje interno.	Reintento con cola y registro del evento fallido. +1
EX-INT-001	Servicio externo no disponible	Proveedor, Admin, Sistema	Un servicio SOA dependiente no responde.	Responder 503, aplicar timeout y política de retry controlado.
EX-DATA-001	Desincronización de datos	Admin, Vendedor, Proveedor	Inventario, pedidos o pagos difieren entre sistemas.	Marcar excepción de conciliación y lanzar proceso batch/manual de ajuste.

EX-AUD-001	Operación bajo revisión	Administrador	La transacción entra en disputa, validación o auditoría.	Congelar la operación y registrar evidencia.
EX-SYS-001	Error interno no controlado	Todos	Falla inesperada de servidor o lógica. +1	Responder 500 con correlationId, sin exponer stack trace. +1

3.2.4 Validaciones

El sistema deberá aplicar mecanismos de validación sobre los datos ingresados y sobre la lógica de procesamiento implementada, con el propósito de garantizar coherencia entre las entradas, las transformaciones realizadas y las salidas generadas. Estas validaciones deberán verificar que la información cumpla con criterios de integridad, consistencia, formato, obligatoriedad y correspondencia con las reglas de negocio establecidas.

De igual manera, el sistema deberá asegurar que toda transformación de datos tenga un sentido funcional verificable, de modo que el resultado obtenido sea medible, interpretable y útil para el actor que lo consulta o consume. En consecuencia, el despliegue de información deberá reflejar con precisión el estado real de la operación ejecutada, manteniendo trazabilidad y claridad en cada proceso.

Puntos que detallar para la validación:

- Validación de campos obligatorios, tipo de dato, longitud, formato y rango.
- Validación de reglas de negocio antes de confirmar operaciones.
- Validación de consistencia entre datos relacionados, por ejemplo stock, pedido, precio y pago.
- Validación de transformación de datos antes de mostrarlos o enviarlos a otro servicio.
- Validación de salida para asegurar precisión, integridad y trazabilidad.

3.3. Implementación

3.3.1. Tecnologías clave

El sistema contempla la integración de diversas tecnologías orientadas al desarrollo, análisis, despliegue y operación de la solución. Para la visualización de datos, se prevé el uso de D3.js en componentes gráficos interactivos y de Power BI para la elaboración de dashboards e indicadores analíticos. En el ámbito de almacenamiento de información, PostgreSQL será utilizado como sistema gestor de base de datos principal, mientras que Firebase podrá considerarse como una tecnología complementaria en fases de despliegue o en funcionalidades que requieren sincronización en tiempo real.

Respecto a la arquitectura del sistema, Spring Boot será empleado para la implementación del backend, mientras que TypeScript permitirá estructurar el frontend bajo un enfoque robusto y mantenible. De forma complementaria, se analizará la posible incorporación de herramientas vinculadas a inteligencia artificial, como

Putter.js, según las necesidades funcionales y la evolución del proyecto. Por otro lado, Git y GitHub se utilizarán como soporte para el control de versiones y la gestión del desarrollo colaborativo. Finalmente, Heroku se proyecta como una alternativa para el despliegue futuro del sistema en un entorno dinámico, e Izipay se establece como la solución integral para el procesamiento de pagos, reemplazando integraciones independientes con Yape y Plin.

3.3.2. APIs

Para la gestión de pagos del sistema, se considera la integración de Izipay como pasarela de pago principal, dado que proporciona una infraestructura especializada para el procesamiento seguro de transacciones electrónicas. Su incorporación permitirá centralizar los cobros realizados dentro de la plataforma y establecer un mecanismo uniforme para la autorización, validación y notificación de pagos vinculados a las operaciones comerciales del sistema.

Asimismo, Izipay ofrece una API de integración orientada a aplicaciones web y móviles, incluyendo funcionalidades como generación de formToken, validación de firma e IPN, las cuales permiten mantener comunicación segura entre el sistema y la plataforma de pagos. En este sentido, su uso resulta adecuado para integrarse con la arquitectura backend propuesta y para garantizar trazabilidad en las transacciones económicas gestionadas por la solución. La lógica recomendada desde nosotros para realizar vía Izipay es:

- El cliente mayorista crea un pedido B2B.
- El sistema valida stock, precios, disponibilidad de puestos y reglas comerciales antes de permitir pagar.
- El Servicio de Pagos genera un intento de pago con orderId único y lo registra como PENDIENTE.
- Izipay devuelve formToken y publicKey; el frontend abre la pasarela.
- Izipay procesa el pago y luego tu backend valida firma y recibe IPN/Webhook.
- Si el pago es aprobado, el pedido cambia a PAGADO o CONFIRMADO_PARA_DESPACHO; si falla, queda en PAGO_RECHAZADO o PENDIENTE_REINTENTO.
- Todo el flujo debe quedar auditado con orderId, transactionId, usuario, timestamp y servicio origen.

Lógica de estados para Pedidos:

- BORRADOR: Cliente arma el carrito, valida stock preliminar. Puede editar o descartar.
- PENDIENTE_PAGO: Cliente confirma carrito, sistema valida stock final y genera intento de pago.
- PAGADO: Izipay confirma vía IPN/Webhook, backend actualiza. Aún no confirmado por vendedor.
- CONFIRMADO: Vendedor verifica stock físico y confirma disponibilidad.

- EN_DESPACHO: Proveedor confirma envío o prepara mercancía.
- ENTREGADO: Cliente recibe y confirma recepción.
- CANCELADO: Rechazado en cualquier etapa antes de entrega (cliente, vendedor o admin).
- EN_DISPUTA: Admin interviene por inconsistencia, devolución o queja.

Transiciones clave (ejemplos):

- BORRADOR a PENDIENTE_PAGO: Cliente hace "Pagar ahora".
- PENDIENTE_PAGO a PAGADO: Solo por validación backend de Izipay (no por frontend).
- PAGADO a CONFIRMADO: Vendedor chequea stock real y aprueba.
- CONFIRMADO a CANCELADO: Si stock no alcanza físicamente.
- Cualquier estado a EN_DISPUTA: Admin detecta anomalía o reclamo.

La regla de oro es que ningún estado cambia por respuesta del navegador; solo por eventos verificados en servicios backend (inventario, pagos, auditoría).

Lógica de estados para Pagos:

- CREADO: Backend genera formToken y registra intento en DB.
- PENDIENTE: Cliente interactúa con formulario de Izipay.
- APROBADO: Izipay envía IPN/Webhook con firma válida, backend verifica y confirma.
- RECHAZADO: Izipay rechaza (fondos insuficientes, tarjeta inválida).
- EXPIRADO: Sesión de pago supera 30 minutos sin completar.
- EN_CONCILIACION: Pago "visualmente OK" pero sin IPN o comprobante; requiere verificación manual.

Transiciones clave:

- PENDIENTE a APROBADO: IPN validado + firma OK + monto coincide.
- PENDIENTE a RECHAZADO: Izipay devuelve error.
- APROBADO a EN_CONCILIACION: Sin comprobante o monto no cuadra con pedido.
- EN_CONCILIACION a APROBADO: Admin confirma manualmente.

Otra regla de oro es el pedido no pasa a PAGADO hasta que pago llegue a APROBADO en backend. Si el cliente ve "pago exitoso" pero IPN falla, queda en conciliación.

3.3.3. Dependencias externas

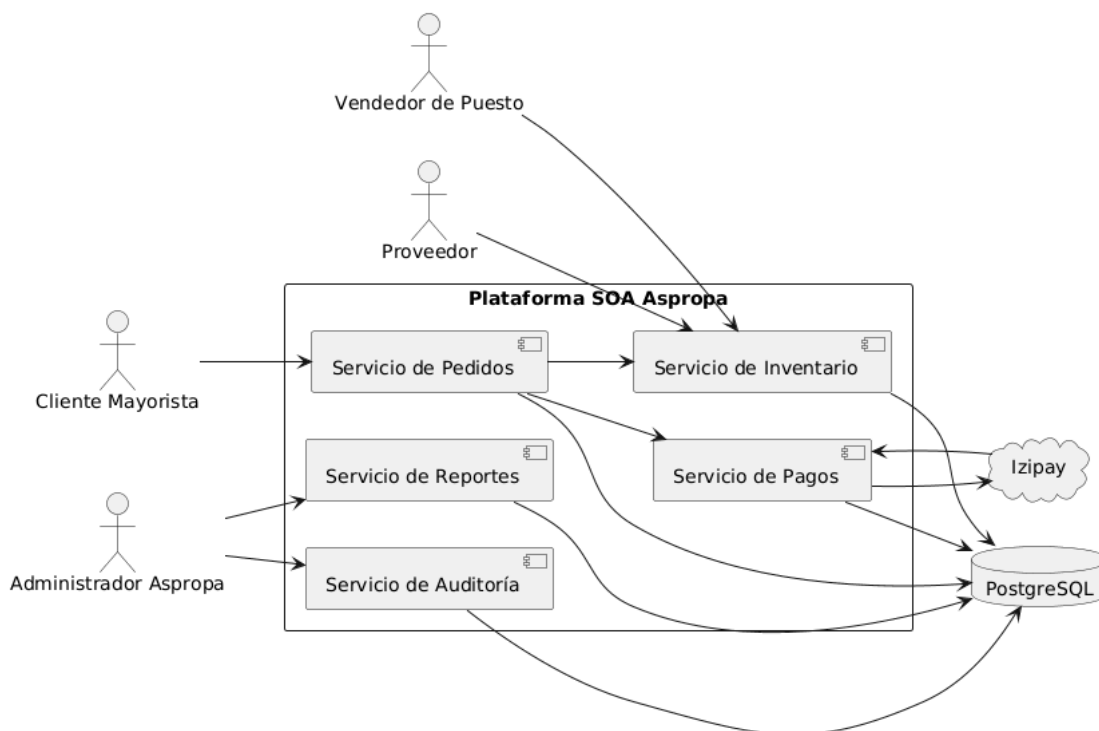
Las dependencias externas del sistema comprenden aquellos servicios, plataformas, librerías y herramientas de terceros que apoyan la operación, integración, análisis, despliegue y mantenimiento de la solución, sin formar parte de la lógica central del negocio. Estas dependencias permiten extender las capacidades del sistema en áreas como procesamiento de pagos, almacenamiento de datos, visualización analítica, control de versiones, despliegue en la nube e interoperabilidad con servicios externos.

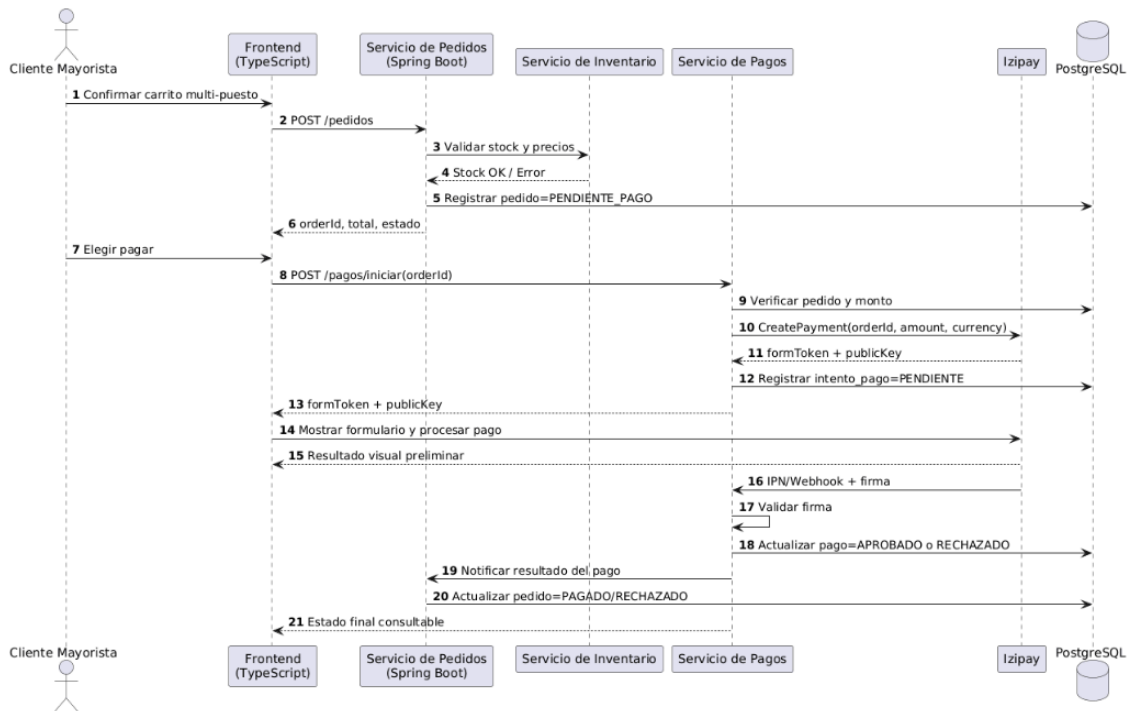
Su identificación es importante porque cada dependencia externa introduce condiciones de uso, restricciones técnicas, costos potenciales, políticas de disponibilidad y consideraciones de seguridad que deben ser evaluadas dentro de la arquitectura SOA propuesta. Por ello, cada una debe documentarse indicando su propósito, alcance, tipo de integración, criticidad operativa y posible impacto en caso de falla o cambio del proveedor.

3.3.4. Aspectos transversales

En el flujo de pagos, los aspectos transversales se manifiestan en la necesidad de aplicar validaciones previas del pedido, mecanismos de idempotencia para evitar cobros duplicados, trazabilidad basada en identificadores únicos, auditoría de cada intento de pago y manejo controlado de excepciones cuando la pasarela no responde o devuelve resultados inconsistentes. La integración con Izipay debe diseñarse bajo estos principios para asegurar que el procesamiento financiero no dependa únicamente del frontend, sino de una confirmación segura y verificable desde los servicios internos del sistema.

En consecuencia, el Servicio de Pagos actúa como punto de control entre la lógica del negocio y la pasarela externa, encapsulando la creación de sesiones de pago, la verificación de firmas, la recepción de notificaciones IPN y la actualización de estados del pedido. Este enfoque mantiene desacoplamiento, trazabilidad y gobernanza dentro de la arquitectura SOA, evitando que la dependencia externa introduzca inconsistencias en los procesos de inventario, pedidos y liquidaciones.





3.4 Lógica de negocio

3.4.1. Reglas de negocio

Las reglas de negocio del sistema establecen que toda operación comercial deberá ejecutarse únicamente bajo condiciones válidas de autorización, disponibilidad, consistencia e integridad de datos. En consecuencia, el procesamiento de pedidos, inventarios, pagos y reportes estará sujeto al cumplimiento de restricciones previamente definidas sobre stock, estado de actores, condiciones de pago, precios vigentes, trazabilidad y auditoría.

Asimismo, el sistema deberá asegurar que las operaciones críticas sean independientes, que las actualizaciones de inventario se reflejen en tiempo real y que cualquier modificación relevante en pedidos, precios o estados de pago quede debidamente registrada. Estas reglas permiten reducir inconsistencias, prevenir duplicidades y mantener control operacional sobre los servicios expuestos en la arquitectura SOA.

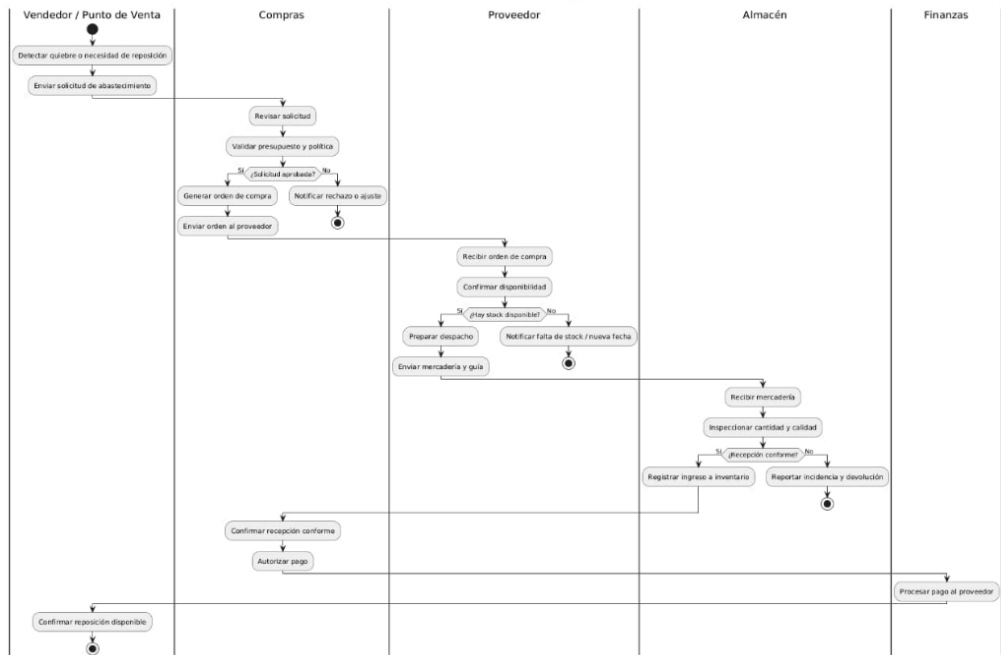
Proceso	Reglas de negocio sugeridas
Registro de actores	Validación de rol, identidad comercial, estado activo y permisos de acceso.
Inventario	Stock no negativo, actualización inmediata, control de concurrencia e idempotencia.
Pedidos B2B	Validación de stock, estado del proveedor, precio vigente, monto mínimo y estado del pedido.
Pagos	Confirmación única, conciliación con pasarela, emisión de comprobante y trazabilidad.
Auditoría	Registro obligatorio de eventos, retención de historial y revisión por administrador.

Proceso	Regla de Negocio	Condiciones	Excepciones	Actores
Registro de Actores	1. Solo personas físicas con actividad comercial verificada o personas jurídicas constituidas legalmente pueden registrarse.	Verificación manual por administrador. Estado inicial: pendiente.	Rechazo si datos inconsistentes o actor no calificado.	Administrador, Solicitante
Gestión de Inventario	2. Un vendedor solo puede gestionar el inventario asociado a su puesto específico.	Stock no puede ser negativo. Actualización atómica e idempotente.	Conflicto de concurrencia: reintento con nueva versión.	Vendedor de Puesto
Catálogo de Proveedores	3. El proveedor solo puede publicar productos con precios B2B válidos y stock disponible.	Precios con vigencia definida. Stock sincronizado en tiempo real.	Producto descontinuado: notificación inmediata.	Proveedor
Creación de Pedidos B2B	4. Un pedido B2B debe contener al menos un producto, cantidad mínima de 10 unidades y monto total superior a S/50.	Carrito multi-puesto permitido. Validación cruzada de disponibilidad.	Stock insuficiente: propuesta de parcial o backorder.	Cliente Mayorista, Vendedor
Confirmación de Pedidos	5. Un vendedor solo puede confirmar pedidos cuando tenga stock suficiente y el proveedor esté activo.	Notificación automática al cliente y proveedor al confirmar.	Proveedor inactivo: puesta en cola o rechazo automático.	Vendedor de Puesto, Cliente
Procesamiento de Pagos	6. El pago debe procesarse antes de confirmar envío y solo con métodos autorizados (Izipay).	Conciliación con pasarela de pagos. Comprobante obligatorio.	Pago rechazado: pedido en estado pendiente.	Cliente Mayorista, Sistema
Cancelación y Devoluciones	7. Cancelación solo permitida antes del despacho; devoluciones requieren aprobación del vendedor.	Penalización por cancelaciones frecuentes. Inspección física para devoluciones.	Cancelación no permitida: explicación del estado actual.	Cliente, Vendedor, Proveedor

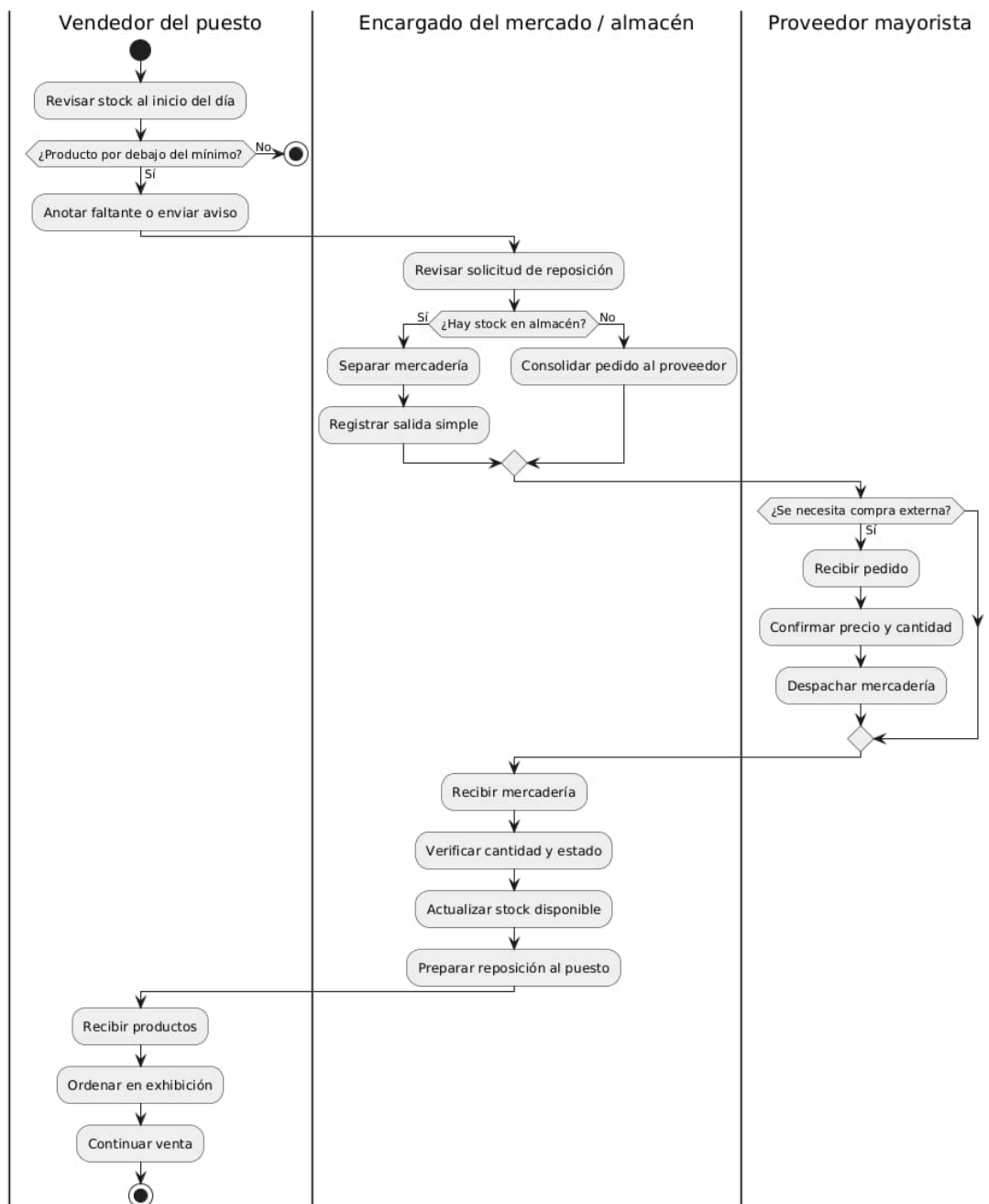
3.4.2. Procesos y flujos

Se han visto 5 procesos críticos en dicho mercado popular:

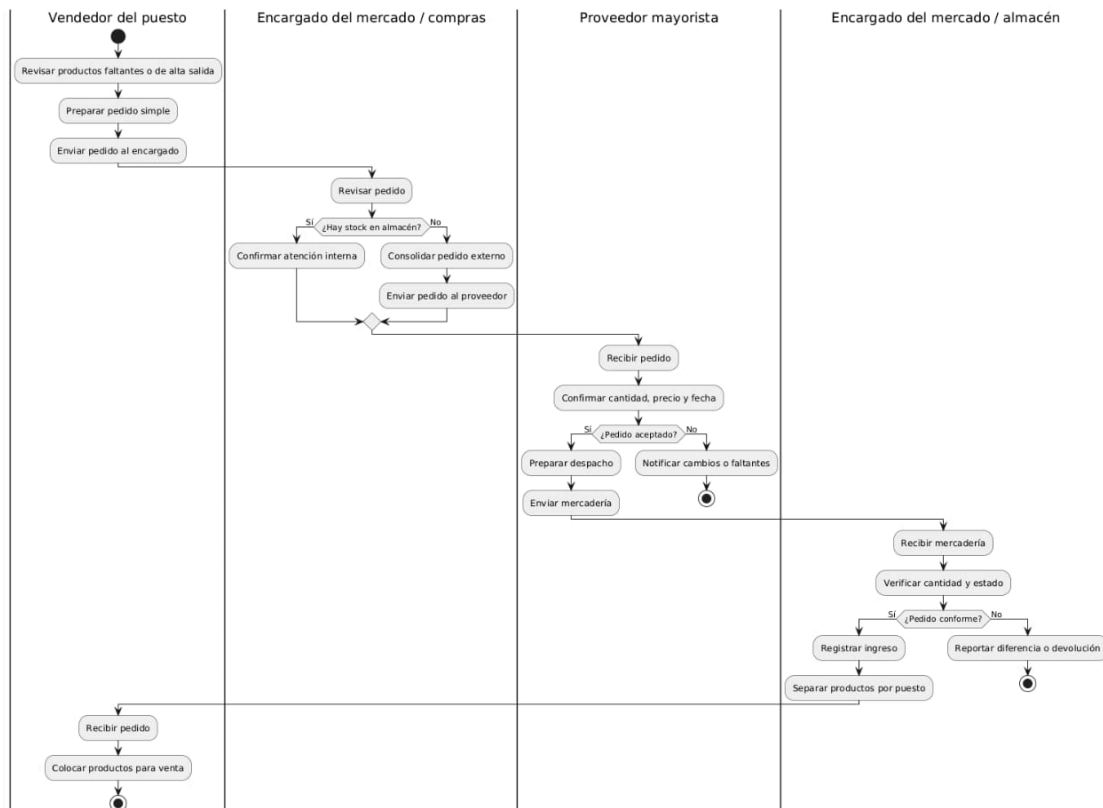
Proceso Crítico I: Abastecimiento y compras B2B



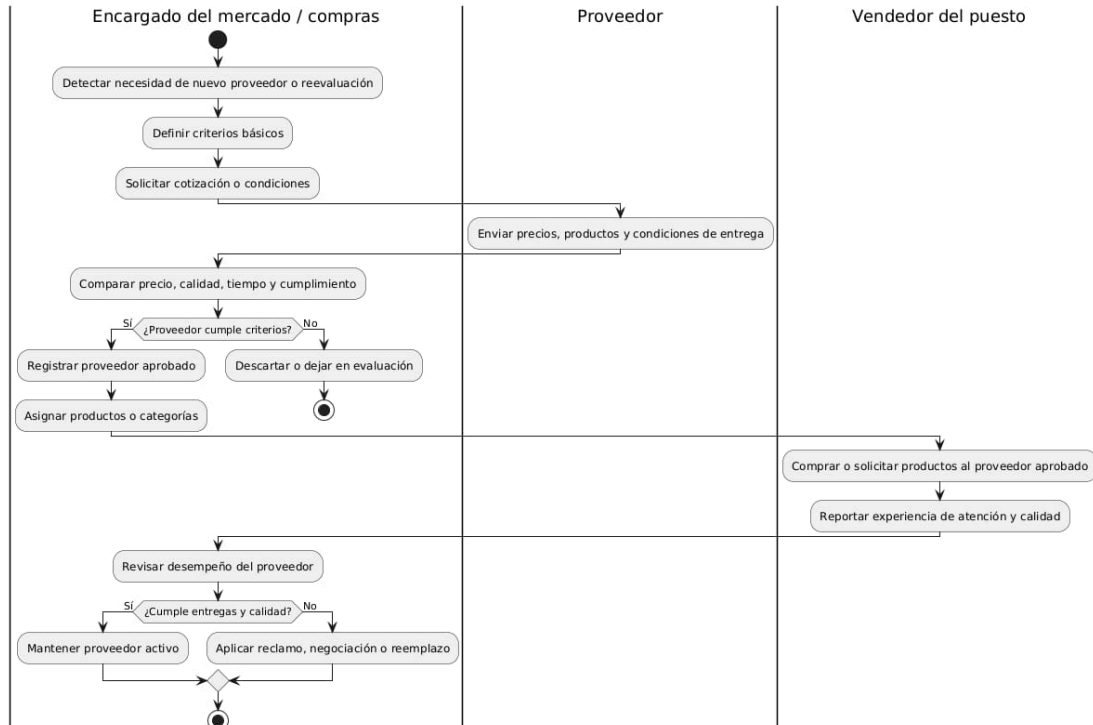
Proceso Crítico II: Reposición por stocks



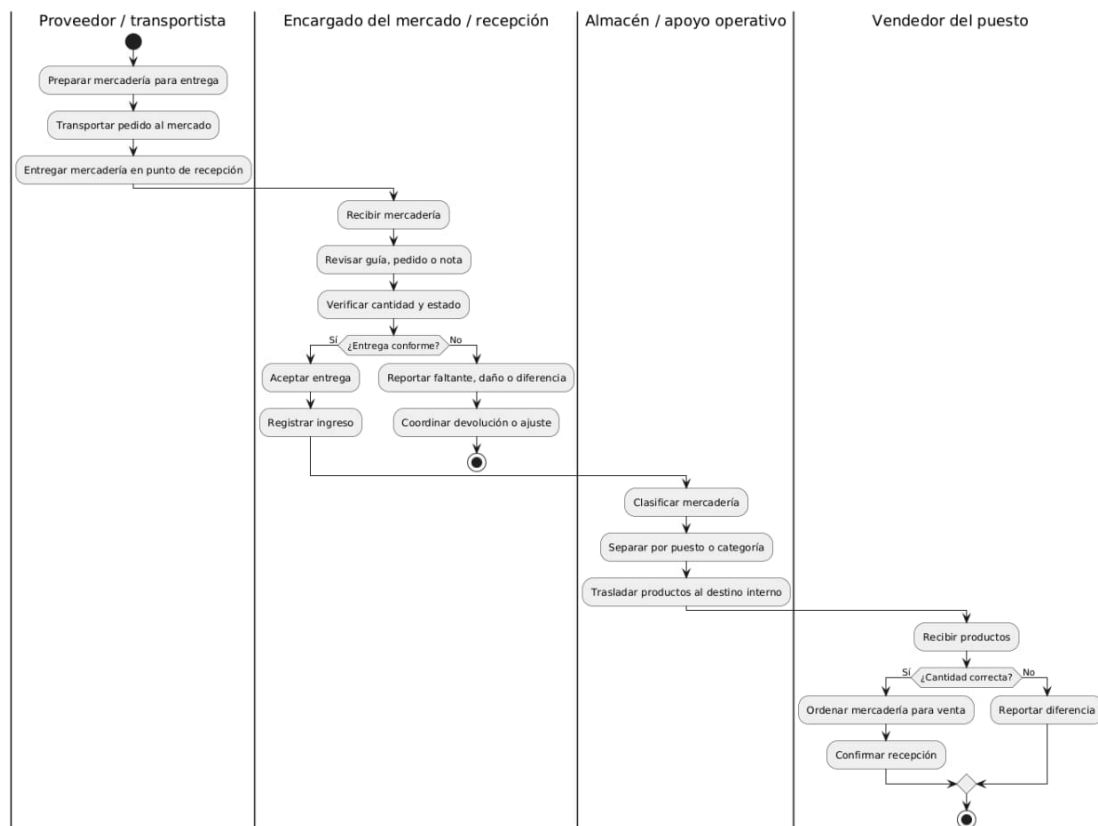
Proceso Crítico III: Gestión de pedidos



Proceso Crítico IV: Gestión de proveedores



Proceso Crítico V: Logística y cumplimiento de entrega



3.4.3. Reglas de validación de negocio

Las reglas de validación del negocio son las verificaciones específicas del dominio que el sistema debe hacer antes de procesar cualquier operación, para que solo se ejecuten acciones que tengan sentido comercial: no vender sin stock, no cobrar sin pedido válido, no confirmar sin pago, etc. En la SOA de Asproa para el B2B establecido, estas reglas se aplican en los servicios de pedidos, inventario y pagos, y ayudan a prevenir errores que generan disputas en un mercado popular. Se diferencian de validaciones técnicas (formato, tipo) porque son reglas del negocio real: cantidades mínimas, estados permitidos, autorizaciones.

Principios para las reglas:

- Verificables: El código las puede comprobar automáticamente.
- Concretas: Números, formatos, condiciones claras (no "stock suficiente", sino "stock >= cantidad").
- Por proceso: Una por operación (registro, pedido, pago).
- Con excepción: Si falla, código específico y acción (rechazar, advertir, escalar).
- Idempotentes: Validaciones que no cambian datos al ejecutarse.

Proceso	Campo	Validación	Regla de Negocio	Excepción si falla
Registro Actor	nombreComercial, ruc, telefono	Nombre 3-50 chars no vacío, RUC 11 dígitos peruanos, tel 9 dígitos.	Debe ser aprobado manual por admin en 24h.	REG-001: Datos de actor inválidos va Estado PENDIENTE.

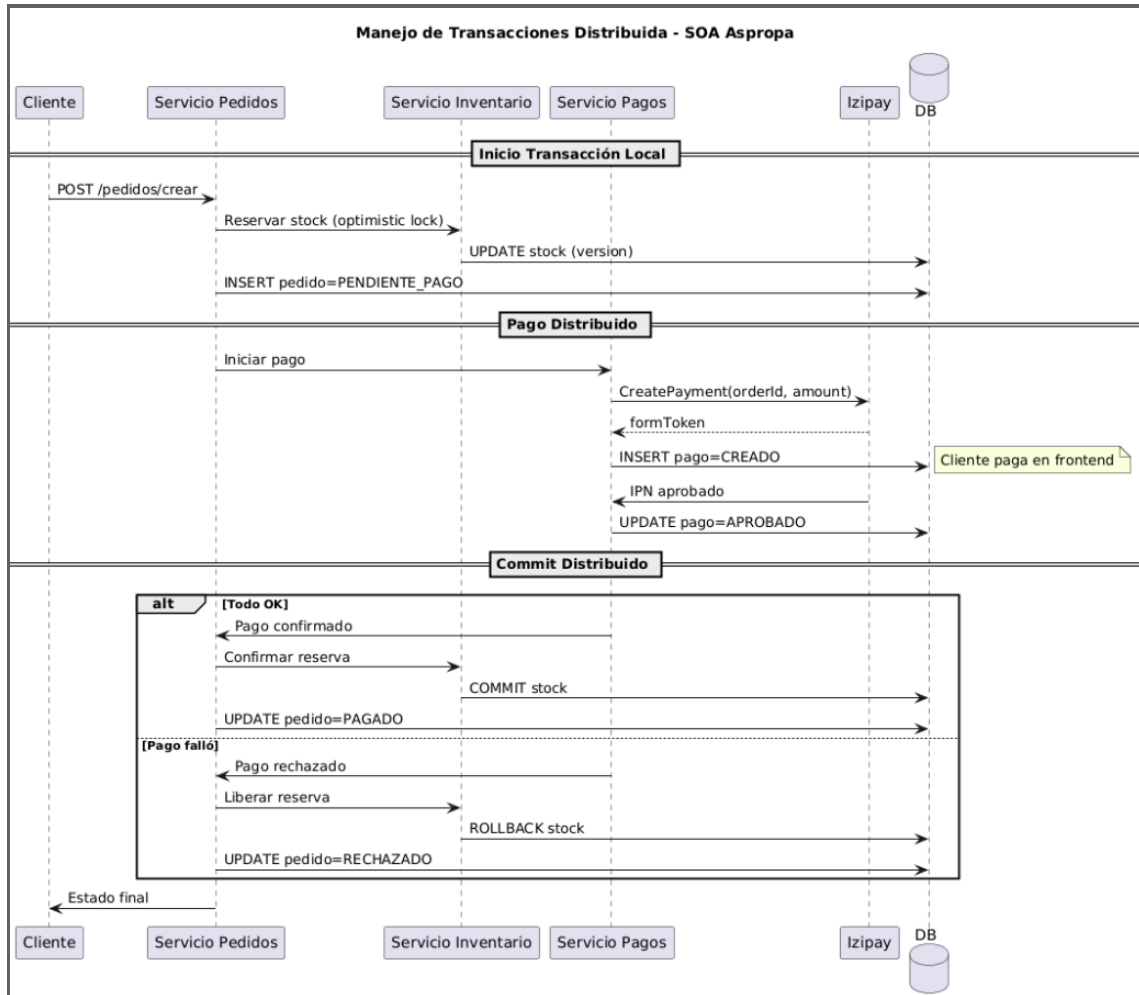
Inventario	cantidad, precioUnitario	Cantidad > 0 entera, precio > 0 decimal 2 cifras, precio <= costoMaximo.	No permitir stock negativo post-venta.	INV-002: Stock insuficiente ergo rechazo + max disponible.
Producto Catálogo	nombre, precioB2B, stock	Nombre 3-100 chars, precioB2B > 0, stock >= 0, precioB2B < precioPublico.	Precio B2B con vigencia <=30 días.	CAT-001: Precio inválido por producto oculto.
Pedido B2B	productos[], cantidades[], puestos[]	Cada producto existe en puesto, sum(cantidades) >=10, montoTotal >= S/50.	Carrito multi-puesto: stock por vendedor.	PED-001: Pedido inválido y el detalle de errores.
Confirmación Pedido	stockDisponible, proveedorActivo	stock >= cantidad, proveedor estado=ACTIVO, pago inicial recibido.	Solo confirma si stock físico OK.	PED-002: No se puede confirmar por motivo específico.
Pago	montoTotal, orderId	monto == totalCalculado, orderId único/no duplicado.	Idempotente por orderId.	PAY-001: Pago rechazado/duplicado para reintento controlado.
Cancelación	motivoCancelacion, estadoPedido	estado en [BORRADOR, PENDIENTE_PAGO, CONFIRMADO], motivo obligatorio.	Penalización si >3/mes.	CAN-001: Cancelación no permitida a un estado actual.
Devolución	motivoDevolucion, inspeccion	Dentro 72h de entrega, vendedor aprueba inspección.	Reembolso solo si OK.	DEV-001: Devolución rechazada y una razón.
Reporte	fechaDesde, fechaHasta, actorId	Fechas válidas (desde <= hasta), actor autorizado para rango.	Trazabilidad completa obligatoria.	REP-001: Acceso denegado por rol insuficiente.

3.4.4. Manejo de transacciones

El sistema adopta un modelo de transacciones distribuidas basado en sagas compensatorias para coordinar operaciones críticas que involucren múltiples servicios SOA y componentes externos como Izipay. Dado que no es posible implementar transacciones ACID tradicionales entre servicios independientes, se emplea un enfoque de consistencia eventual donde cada servicio ejecuta su responsabilidad local y registra mecanismos de compensación reversibles.

En el flujo principal de pedidos B2B, la creación de un pedido implica reserva provisional de stock, generación de intento de pago y confirmación asíncrona vía IPN de Izipay. Si el pago es exitoso, se confirma la reserva y el pedido avanza; en caso contrario, se libera el stock reservado y el pedido se marca como rechazado, manteniendo integridad sin intervención manual.

El diagrama de secuencia adjunto ilustra este mecanismo, destacando el uso de correlationId para idempotencia, estados intermedios persistidos y compensaciones automáticas. Este patrón asegura resiliencia operativa frente a fallos parciales, alineándose con las características de disponibilidad, trazabilidad e idempotencia definidas en los requisitos no funcionales del sistema.



3.4.5. Excepciones de negocio

Las excepciones de negocio representan situaciones operativas previstas donde las reglas comerciales del mercado no permiten continuar el flujo normal, requiriendo intervención específica del actor o escalamiento al administrador. Estas situaciones incluyen stock insuficiente, proveedores inactivos, pagos rechazados, cancelaciones fuera de ventana o límites de uso excedidos, todas comunes en entornos B2B informales como mercados populares.

El sistema responde a estas excepciones con mensajes estructurados que incluyen código único, descripción clara en lenguaje natural, estado resultante del proceso y recomendación de acción, siguiendo el estándar Problem Details para APIs. De esta manera, un vendedor sabe exactamente por qué no puede confirmar un pedido (ej. "PED-002: Proveedor inactivo"), un cliente mayorista entiende que su carrito no cumple mínimo ("PED-001") y el admin puede auditar bloqueos específicos ("AUD-001").

Cada excepción está diseñada para ser retryable o no retryable según corresponda: stock insuficiente permite reintento tras reposición (retryable=true), pero cancelación fuera de plazo requiere acción humana (retryable=false). Además, todas incluyen correlationId para

trazabilidad completa, permitiendo reconstruir el contexto completo de la operación fallida desde logs y auditoría.

```
{
  "type": "https://aspropa.pe/errors/inv-001",
  "title": "Stock insuficiente",
  "status": 409,
  "code": "INV-001",
  "detail": "Producto PAPA (ID:123) solo tiene 15kg disponibles, solicitó 25kg",
  "instance": "/pedidos/ORD-2026-00123/items/0",
  "correlationId": "abc123-def456",
  "retryable": true,
  "suggestion": "Reducir a 15kg o esperar reposición"
}
```

Cód.	Excepción	Proceso	Causa	Acción del sistema	Acción del actor	Estado resultante	HTTP
REG-001	Datos actor inválidos	Registro	RUC malformado, nombre vacío, tel inválido	Rechazar registro, estado PENDIENTE	Corregir datos, reenviar	PENDIENTE_APROBACION	400
INV-001	Stock insuficiente	Inventario/Pedido	cantidad > stockDisponible	Reservar máximo disponible, rechazar exceso	Reducir cantidad o backorder	PARCIAL_RESERVADO	409
INV-002	Concurrencia stock	Inventario	Dos ventas simultáneas	Rechazar con versión actual, forzar relectura	Reintentar con stock actual	INV_CONFLICTO	409
CAT-001	Producto no disponible	Catálogo/Pedido	Producto discontinuado o puesto inactivo	Excluir del carrito, sugerir alternativas	Seleccionar otro producto	PRODUCTO_INVALIDO	404
PED-001	Pedido mínimo no cumplido	Pedido B2B	<10 unidades o <\$/50	Rechazar, mostrar mínimo requerido	Aumentar cantidad/productos	BORRADOR_INVALIDO	400
PED-002	Proveedor inactivo	Confirmación	proveedor.estado != ACTIVO	Pausar pedido, notificar vendedor	Cambiar proveedor o esperar	PENDIENTE_PROVEEDOR	503

PED-003	Precio desactualizado	Confirmación	precioActual != precioPedido	Re-calcular total, pedir reconfirmación	Aceptar nuevo precio o cancelar	PRECIO_CAMBIO	409
PAY-001	Pago rechazado	Pagos	Izipay devuelve error (fondos, tarjeta)	3 reintentos automáticos, luego manual	Nuevo método pago o fondos	PAGO_RECHAZADO	402
PAY-002	Duplicación detectada	Pagos	orderId ya pagado	Devolver estado actual, no procesar	Consultar estado existente	YA_PROCESADO	409
CAN-001	Cancelación no permitida	Cancelación	Estado CONFIRMADO o posterior	Explicar ventana permitida	Devolución formal si aplica	CANCELACION_DENEGADA	403
DEV-001	Devolución rechazada	Devolución	Inspección falló o fuera plazo	Notificar motivo, cerrar caso	Aceptar producto o escalar	DEV_RECHAZADA	403
AUD-001	Operación en revisión	Auditoría	Admin puso EN_DISPUTA	Bloquear modificaciones, notificar	Esperar resolución admin	BLOQUEADO_AUDITORIA	423
SEC-001	Acceso fuera de rol	Todos	Actor intenta operación no autorizada	Log de seguridad, rechazar acceso	Usar credenciales correctas	ACCESO_DENEGADO	403
LIM-001	Límite de uso excedido	Todos	>100 pedidos/hora o 500 consultas/día	Throttling temporal (1h)	Esperar o contactar admin	LUGAR_EXCEDIDO	429

La tabla adjunta detalla las 14 excepciones principales organizadas por proceso, especificando causa comercial, comportamiento del sistema, acción requerida del usuario y código HTTP correspondiente. Cada excepción sigue el estándar Problem Details (RFC 9457), proporcionando trazabilidad completa mediante correlationId y diferenciando claramente entre condiciones retryable y no retryable según su naturaleza transitoria o permanente.

Este catálogo permite al sistema responder de forma predecible y auditable ante situaciones comerciales esperadas, reduciendo significativamente las disputas entre vendedores, proveedores y clientes mayoristas al establecer expectativas claras sobre qué operaciones son válidas y cuáles requieren corrección humana.

El sistema propuesto deberá alinearse con el marco normativo peruano aplicable a plataformas digitales con operaciones comerciales y tratamiento de información personal. En primer término, se deberá observar la Ley N.º 29733, Ley de Protección de Datos Personales, cuyo objeto consiste en garantizar el derecho fundamental a la protección de los datos personales a

través de su tratamiento adecuado, así como su reglamentación actualizada, vigente recientemente.

3.4.6. Cumplimiento Normativo

El sistema propuesto deberá alinearse con el marco normativo peruano aplicable a plataformas digitales con operaciones comerciales y tratamiento de información personal. En primer término, se deberá observar la Ley N.º 29733, Ley de Protección de Datos Personales, cuyo objeto consiste en garantizar el derecho fundamental a la protección de los datos personales a través de su tratamiento adecuado, así como su reglamentación actualizada, vigente desde marzo de 2025. Ello implica implementar medidas orientadas a la licitud del tratamiento, control de acceso, uso proporcional de la información, trazabilidad y resguardo de los datos personales gestionados por la plataforma.

En segundo término, el diseño de la solución deberá considerar la Ley N.º 27269, Ley de Firmas y Certificados Digitales, que otorga validez jurídica a la firma electrónica en condiciones legalmente establecidas. Este aspecto adquiere relevancia para la formalización de actos digitales dentro del sistema, tales como aceptaciones, conformidades, autorizaciones o constancias de operación, particularmente en escenarios donde se requiera fortalecer evidencia, autenticidad e integridad documental.

Adicionalmente, la plataforma deberá adecuarse a los estándares de protección del consumidor aplicables al comercio electrónico, especialmente a partir de la incorporación de reglas que prohíben interfaces o mecanismos digitales que distorsionen la autonomía del usuario y que exigen canales efectivos de reclamo y atención. Bajo este criterio, la arquitectura funcional y la experiencia de usuario deberán preservar la transparencia informativa, la claridad contractual, la libertad de decisión y la posibilidad real de formular reclamos o solicitudes de devolución.

El sistema deberá contemplar las obligaciones tributarias y documentarias vinculadas a la actividad comercial digital, en particular la emisión de comprobantes de pago electrónicos y la conservación de evidencia de las transacciones. Esta exigencia refuerza la necesidad de integrar funciones de trazabilidad, validación y consulta documental que respalden tanto la operación comercial como el cumplimiento administrativo y fiscal de los actores involucrados.

3.5. Data

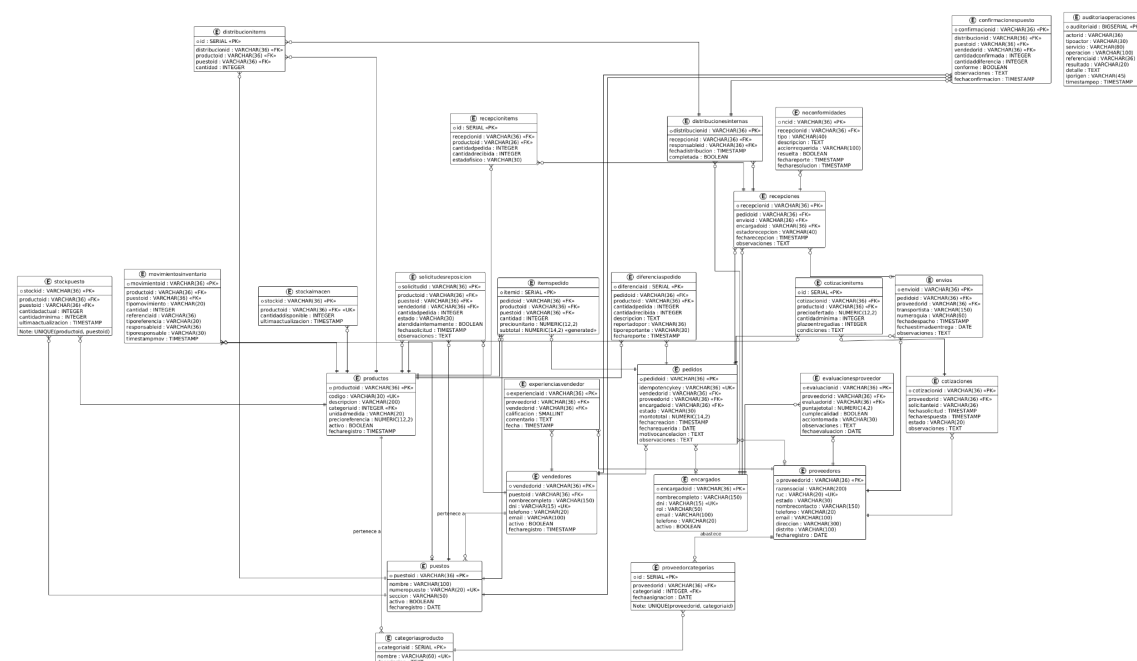
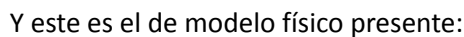
La capa de datos del sistema será responsable de almacenar y gestionar la información estructural y transaccional necesaria para la operación de la plataforma. Entre los principales tipos de datos que se almacenarán se encuentran los datos de actores del sistema, tales como puestos, vendedores, encargados y proveedores; los datos del catálogo de productos y sus categorías; la información de inventario y movimientos de stock; los pedidos B2B y sus detalles; las solicitudes de reposición; los envíos, recepciones y confirmaciones logísticas; así como los registros de pagos, auditoría, incidencias y trazabilidad de las operaciones realizadas.

La base de datos principal que se utilizará será PostgreSQL, debido a que ofrece un modelo relacional sólido, soporte transaccional, integridad referencial y capacidades adecuadas para manejar operaciones críticas del entorno comercial propuesto. Además, el documento ya contempla una arquitectura SQL organizada por dominios funcionales, con tablas, claves primarias, claves foráneas e identificadores únicos que respaldan la consistencia de procesos como inventario, pedidos, y auditoría.

Esta capa se encarga del acceso y persistencia de la información, actuando como el punto donde los servicios almacenan, consultan, actualizan y recuperan datos de manera controlada. Su función no se limita al guardado físico de registros, sino que también asegura la integridad, trazabilidad, consistencia y disponibilidad de la información que consumen las demás capas del sistema, especialmente la lógica de negocio y los servicios expuestos por la arquitectura SOA.

El modelo de datos canónico de Asproa actúa como una capa común de significado para los servicios SOA del sistema, permitiendo que pedidos, inventario, proveedores y logística compartan estructuras homogéneas sin acoplarse a implementaciones internas específicas. Este enfoque reduce transformaciones innecesarias entre servicios y facilita la interoperabilidad, la reutilización y la evolución controlada del sistema. La base de datos en SQL materializa ese modelo mediante una arquitectura relacional organizada por dominios funcionales: actores, catálogo, stock, pedidos, proveedores, logística y auditoría. La presencia de tablas especializadas, claves, relaciones e identificadores únicos como `idempotency_key` refuerza la consistencia del negocio y soporta las operaciones críticas del entorno B2B.

Este es el esquema de modelo lógico con varias categorías:



3.5.3. Diccionario de datos

Presenta diccionarios y ejecución de esto:

Sección 1: Actores del sistema

Campo	Tipo	Nulable	Clave	Descripción funcional
puesto_id	VARCHAR(36)	NO	PK	UUID autogenerado. Identifica unívocamente cada puesto
nombre	VARCHAR(100)	NO	—	Nombre comercial del puesto
numero_puesto	VARCHAR(20)	NO	UNIQUE	Código alfanumérico con patrón A-12 (sección + número)
seccion	VARCHAR(50)	SÍ	—	Pasillo o zona del mercado
activo	BOOLEAN	NO	—	TRUE si el puesto está operativo. Default: TRUE
fecha_registro	DATE	NO	—	Fecha de ingreso al sistema. Default: fecha actual

Campo	Tipo	Nulable	Clave	Descripción funcional
vendedor_id	VARCHAR(36)	NO	PK	UUID del vendedor
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto al que está asignado
nombre_completo	VARCHAR(150)	NO	—	Nombres y apellidos completos
dni	VARCHAR(15)	NO	UNIQUE	DNI peruano. Único en el sistema
telefono	VARCHAR(20)	SÍ	—	Número de contacto
email	VARCHAR(100)	SÍ	—	Correo electrónico
activo	BOOLEAN	NO	—	Estado en el sistema. Default: TRUE
fecha_registro	TIMESTAMP	NO	—	Fecha y hora de registro

Campo	Tipo	Nulable	Clave	Descripción funcional
encargado_id	VARCHAR(36)	NO	PK	UUID del encargado
nombre_completo	VARCHAR(150)	NO	—	Nombres y apellidos completos

dni	VARCHAR(15)	NO	UNIQUE	DNI peruano. Único en el sistema
rol	VARCHAR(50)	NO	—	RECEPCION, ALMACEN, COMPRAS o ADMINISTRADOR
email	VARCHAR(100)	SÍ	—	Correo institucional
telefono	VARCHAR(20)	SÍ	—	Teléfono de contacto
activo	BOOLEAN	NO	—	Estado operativo. Default: TRUE

Sección 2: Catálogo de productos

Campo	Tipo	Nulable	Clave	Descripción funcional
categoria_id	SERIAL	NO	PK	Identificador secuencial autoincremental
nombre	VARCHAR(60)	NO	UNIQUE	Nombre del rubro. Ej: ABARROTES, FRUTAS_VERDURAS
descripcion	TEXT	SÍ	—	Descripción ampliada de la categoría

Campo	Tipo	Nulable	Clave	Descripción funcional
producto_id	VARCHAR(36)	NO	PK	UUID del producto
codigo	VARCHAR(30)	NO	UNIQUE	Código interno de identificación
descripcion	VARCHAR(200)	NO	—	Descripción comercial del producto
categoria_id	INTEGER	NO	FK a categorias_producto	Rubro al que pertenece
unidad_medida	VARCHAR(20)	NO	—	UNIDAD, KG, LITRO, etc. Default: UNIDAD
precio_referencia	NUMERIC(12,2)	SÍ	—	Precio de referencia del mercado (orientativo)
activo	BOOLEAN	NO	—	Disponible para pedidos. Default: TRUE
fecha_registro	TIMESTAMP	NO	—	Fecha de ingreso al catálogo

Sección 3: Inventario

Campo	Tipo	Nulable	Clave	Descripción funcional
stock_id	VARCHAR(36)	NO	PK	UUID del registro de stock
producto_id	VARCHAR(36)	NO	FK a productos	Producto cuyo stock se registra
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto propietario del stock
cantidad_actual	INTEGER	NO	—	Unidades disponibles. Mínimo: 0
cantidad_minima	INTEGER	NO	—	Umbral que activa solicitud de reposición. Mínimo: 0
ultima_actualizacion	TIMESTAMP	NO	—	Última modificación del registro

Campo	Tipo	Nulable	Clave	Descripción funcional
stock_id	VARCHAR(36)	NO	PK	UUID del registro
producto_id	VARCHAR(36)	NO	FK a productos, UNIQUE	Un solo registro por producto en almacén
cantidad_disponible	INTEGER	NO	—	Unidades en almacén central. Mínimo: 0
ultima_actualizacion	TIMESTAMP	NO	—	Fecha de última modificación

Campo	Tipo	Nulable	Clave	Descripción funcional
solicitud_id	VARCHAR(36)	NO	PK	UUID de la solicitud
producto_id	VARCHAR(36)	NO	FK a productos	Producto que se solicita reponer
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto que origina la solicitud
vendedor_id	VARCHAR(36)	NO	FK a vendedores	Vendedor que registra la solicitud
cantidad_pedida	INTEGER	NO	—	Unidades solicitadas. Debe ser > 0

estado	VARCHAR(30)	NO	—	PENDIENTE, ATENDIDA_INTERNA, DERIVADA_PROVEEDOR, COMPLETADA, CANCELADA
atendida_interamente	BOOLEAN	SÍ	—	TRUE si se atendió desde almacén propio, FALSE si se derivó a proveedor
fecha_solicitud	TIMESTAMP	NO	—	Fecha y hora de generación
observaciones	TEXT	SÍ	—	Comentarios adicionales del vendedor

Campo	Tipo	Nulable	Clave	Descripción funcional
movimiento_id	VARCHAR(36)	NO	PK	UUID del movimiento
producto_id	VARCHAR(36)	NO	FK a productos	Producto afectado
puesto_id	VARCHAR(36)	SÍ	FK a puestos	Puesto involucrado (NULL en movimientos de almacén)
tipo_movimiento	VARCHAR(20)	NO	—	ENTRADA, SALIDA, AJUSTE, DEVOLUCION
cantidad	INTEGER	NO	—	Unidades del movimiento. Debe ser > 0
referencia_id	VARCHAR(36)	SÍ	—	ID de la entidad que origina el movimiento
tipo_referencia	VARCHAR(30)	SÍ	—	PEDIDO, SOLICITUD o RECEPCION
responsable_id	VARCHAR(36)	NO	—	ID del actor ejecutor (polimórfico, sin FK formal)
tipo_responsable	VARCHAR(30)	NO	—	VENDEDOR o ENCARGADO
timestamp_mov	TIMESTAMP	NO	—	Marca de tiempo del movimiento

Sección 4: Proveedores

Campo	Tipo	Nulable	Clave	Descripción funcional
proveedor_id	VARCHAR(36)	NO	PK	UUID del proveedor
razon_social	VARCHAR(200)	NO	—	Nombre legal de la empresa proveedora

ruc	VARCHAR(20)	NO	UNIQUE	RUC de SUNAT (11 dígitos en Perú)
estado	VARCHAR(30)	NO	—	EN_EVALUACION, APROBADO, ACTIVO, SUSPENDIDO, DESCARTADO
nombre_contacto	VARCHAR(150)	SÍ	—	Nombre del representante comercial
telefono	VARCHAR(20)	SÍ	—	Teléfono del proveedor
email	VARCHAR(100)	SÍ	—	Correo de contacto
direccion	VARCHAR(300)	SÍ	—	Dirección fiscal o comercial
distrito	VARCHAR(100)	SÍ	—	Distrito (contexto Lima / Perú)
fecha_registro	DATE	NO	—	Fecha de incorporación al sistema

Campo	Tipo	Nulable	Clave	Descripción funcional
id	SERIAL	NO	PK	Identificador secuencial
proveedor_id	VARCHAR(36)	NO	FK a proveedores	Proveedor autorizado
categoria_id	INTEGER	NO	FK a categorias_producto	Categoría asignada
fecha_asignacion	DATE	NO	—	Fecha en que se asignó la categoría

Campo	Tipo	Nulable	Clave	Descripción funcional
cotizacion_id	VARCHAR(36)	NO	PK	UUID de la cotización
proveedor_id	VARCHAR(36)	NO	FK a proveedores	Proveedor cotizante
solicitante_id	VARCHAR(36)	NO	—	ID del encargado solicitante (sin FK formal — documentar)
fecha_solicitud	TIMESTAMP	NO	—	Fecha de envío de la solicitud
fecha_respuesta	TIMESTAMP	SÍ	—	Fecha de respuesta del proveedor

estado	VARCHAR(20)	NO	—	ENVIADA, RESPONDIDA, ACEPTADA, RECHAZADA, VENCIDA
observaciones	TEXT	SÍ	—	Notas del proceso

Campo	Tipo	Nulable	Clave	Descripción funcional
id	SERIAL	NO	PK	Identificador secuencial
cotizacion_id	VARCHAR(36)	NO	FK a cotizaciones	Cotización a la que pertenece
producto_id	VARCHAR(36)	NO	FK a productos	Producto cotizado
precio_ofertado	NUMERIC(12,2)	SÍ	—	Precio unitario ofrecido por el proveedor
cantidad_minima	INTEGER	SÍ	—	Cantidad mínima para ese precio
plazo_entrega_dias	INTEGER	SÍ	—	Días hábiles estimados de entrega
condiciones	TEXT	SÍ	—	Condiciones comerciales adicionales

Campo	Tipo	Nulable	Clave	Descripción funcional
evaluacion_id	VARCHAR(36)	NO	PK	UUID de la evaluación
proveedor_id	VARCHAR(36)	NO	FK a proveedores	Proveedor evaluado
evaluador_id	VARCHAR(36)	NO	FK a encargados	Encargado que evalúa
puntaje_total	NUMERIC(4,2)	SÍ	—	Puntaje global de 0 a 10
cumple_calidad	BOOLEAN	NO	—	Si cumple estándares mínimos de calidad
accion_tomada	VARCHAR(30)	SÍ	—	MANTENER, APLICAR_RECLAMO, NEGOCIAR, REEMPLAZAR
observaciones	TEXT	SÍ	—	Comentarios del evaluador
fecha_evaluacion	DATE	NO	—	Fecha de la evaluación

Campo	Tipo	Nulable	Clave	Descripción funcional
id	SERIAL	NO	PK	Identificador secuencial
evaluacion_id	VARCHAR(36)	NO	FK a evaluaciones_proveedor	Evaluación a la que pertenece
criterio	VARCHAR(100)	NO	—	precio, calidad, puntualidad, cumplimiento
puntaje	NUMERIC(4, 2)	NO	—	Puntaje del criterio de 0 a 10
comentario	TEXT	SÍ	—	Justificación del puntaje

Campo	Tipo	Nulable	Clave	Descripción funcional
experiencia_id	VARCHAR(36)	NO	PK	UUID de la experiencia
proveedor_id	VARCHAR(36)	NO	FK a proveedores	Proveedor calificado
vendedor_id	VARCHAR(36)	NO	FK a vendedores	Vendedor que reporta
calificacion	SMALLINT	NO	—	Escala de 1 a 5 (tipo Likert)
comentario	TEXT	SÍ	—	Opinión libre del vendedor
fecha	TIMESTAMP	NO	—	Fecha y hora del registro

Sección 5: Pedidos

Campo	Tipo	Nulable	Clave	Descripción funcional
pedido_id	VARCHAR(36)	NO	PK	UUID del pedido
idempotency_key	VARCHAR(36)	NO	UNIQUE	Clave UUID v4 para evitar duplicados por reintentos de red
vendedor_id	VARCHAR(36)	NO	FK a vendedores	Vendedor que origina el pedido
proveedor_id	VARCHAR(36)	SÍ	FK a proveedores	Proveedor destino (NULL si es pedido interno)

encargado_id	VARCHAR(36)	SÍ	FK a encargados	Encargado que gestiona o aprueba
estado	VARCHAR(30)	NO	—	PENDIENTE, CONFIRMADO, EN_PREPARACION, ENVIADO, RECIBIDO_PARCIAL, ENTREGADO, CANCELADO, CON_DIFERENCIA
monto_total	NUMERIC(14,2)	SÍ	—	Suma de los ítems del pedido
fecha_creacion	TIMESTAMP	NO	—	Fecha y hora de creación
fecha_requerida	DATE	SÍ	—	Fecha máxima de entrega requerida
motivo_cancelacion	TEXT	SÍ	—	Razón de cancelación, si aplica
solicitante_cancelacion	VARCHAR(36)	SÍ	—	ID del actor que canceló
observaciones	TEXT	SÍ	—	Notas adicionales del pedido

Campo	Tipo	Nulable	Clave	Descripción funcional
item_id	SERIAL	NO	PK	Identificador secuencial
pedido_id	VARCHAR(36)	NO	FK a pedidos	Pedido al que pertenece
producto_id	VARCHAR(36)	NO	FK a productos	Producto solicitado
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto destino del producto
cantidad	INTEGER	NO	—	Unidades solicitadas. Debe ser > 0
precio_unitario	NUMERIC(12,2)	NO	—	Precio pactado por unidad. Debe ser >= 0
subtotal	NUMERIC(14,2)	Generado	—	$\text{cantidad} \times \text{precio_unitario}$ — calculado automáticamente (STORED)

Campo	Tipo	Nulable	Clave	Descripción funcional
diferencia_id	SERIAL	NO	PK	Identificador secuencial

pedido_id	VARCHAR(36)	NO	FK a pedidos	Pedido con diferencia
producto_id	VARCHAR(36)	NO	FK a productos	Producto con discrepancia
cantidad_pedida	INTEGER	NO	—	Cantidad originalmente pedida
cantidad_recibida	INTEGER	NO	—	Cantidad efectivamente recibida
descripcion	TEXT	SÍ	—	Descripción de la discrepancia
reportado_por	VARCHAR(36)	NO	—	ID del actor que reporta (polimórfico)
tipo_reportante	VARCHAR(30)	NO	—	VENDEDOR o ENCARGADO
fecha_reporte	TIMESTAMP	NO	—	Fecha y hora del reporte

Sección 6: Logística y Recepción

Campo	Tipo	Nulable	Clave	Descripción funcional
envio_id	VARCHAR(36)	NO	PK	UUID del envío
pedido_id	VARCHAR(36)	NO	FK a pedidos	Pedido que origina el envío
proveedor_id	VARCHAR(36)	NO	FK a proveedores	Proveedor que despacha
transportista	VARCHAR(150)	NO	—	Nombre del transportista o empresa logística
numero_guia	VARCHAR(60)	NO	—	Número de guía de remisión
fecha_despacho	TIMESTAMP	NO	—	Fecha y hora de despacho desde origen
fecha_estimada_entrega	DATE	SÍ	—	Fecha estimada de llegada al mercado
observaciones	TEXT	SÍ	—	Notas del transportista o proveedor

Campo	Tipo	Nulable	Clave	Descripción funcional
id	SERIAL	NO	PK	Identificador secuencial

repcion_id	VARCHAR(36)	NO	FK a recepciones	Recepción a la que pertenece
producto_id	VARCHAR(36)	NO	FK a productos	Producto verificado
cantidad_pedida	INTEGER	NO	—	Cantidad esperada según el pedido
cantidad_recibida	INTEGER	NO	—	Cantidad efectivamente contada
estado_fisico	VARCHAR(30)	SÍ	—	BUENO, DETERIORADO, VENCIDO

Campo	Tipo	Nulable	Clave	Descripción funcional
nc_id	VARCHAR(36)	NO	PK	UUID de la no conformidad
repcion_id	VARCHAR(36)	NO	FK a recepciones	Recepción donde se detectó
tipo	VARCHAR(40)	NO	—	FALTANTE, DANIO, DIFERENCIA_PRECIO, PRODUCTO_INCORRECTO
descripcion	TEXT	NO	—	Descripción detallada del problema
accion_requerida	VARCHAR(100)	SÍ	—	Acción solicitada al proveedor o transportista
resuelta	BOOLEAN	NO	—	Si fue resuelta. Default: FALSE
fecha_reporte	TIMESTAMP	NO	—	Fecha de registro del incidente
fecha_resolucion	TIMESTAMP	SÍ	—	Fecha de cierre

Campo	Tipo	Nulable	Clave	Descripción funcional
id	SERIAL	NO	PK	Identificador secuencial
distribucion_id	VARCHAR(36)	NO	FK a distribuciones_internas	Distribución a la que pertenece
producto_id	VARCHAR(36)	NO	FK a productos	Producto a distribuir
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto destino

cantidad	INTEGER	NO	—	Unidades asignadas al puesto. Debe ser > 0
----------	---------	----	---	--

Campo	Tipo	Nulable	Clave	Descripción funcional
confirmacion_id	VARCHAR(36)	NO	PK	UUID de la confirmación
distribucion_id	VARCHAR(36)	NO	FK a distribuciones_internas	Distribución que generó la entrega
puesto_id	VARCHAR(36)	NO	FK a puestos	Puesto que confirma
vendedor_id	VARCHAR(36)	NO	FK a vendedores	Vendedor que firma la conformidad
cantidad_confirmada	INTEGER	NO	—	Unidades efectivamente recibidas
cantidad_diferencia	INTEGER	NO	—	Diferencia entre asignado y recibido. Default: 0
conforme	BOOLEAN	NO	—	TRUE si acepta sin observaciones
observaciones	TEXT	SÍ	—	Comentarios del vendedor
fecha_confirmacion	TIMESTAMP	NO	—	Fecha y hora de la confirmación

Sección 7: Auditoría transversal

Campo	Tipo	Nulable	Clave	Descripción funcional
auditoria_id	BIGSERIAL	NO	PK	Identificador secuencial de alta capacidad
actor_id	VARCHAR(36)	NO	—	ID del actor que ejecuta la operación
tipo_actor	VARCHAR(30)	NO	—	VENDEDOR, ENCARGADO, PROVEEDOR, SISTEMA
servicio	VARCHAR(80)	NO	—	Nombre del servicio SOA invocado (ej. GestionPedidos)
operacion	VARCHAR(100)	NO	—	Operación ejecutada (ej. CrearPedido)
referencia_id	VARCHAR(36)	SÍ	—	ID de la entidad afectada
resultado	VARCHAR(20)	NO	—	EXITO, ERROR, RECHAZADO

detalle	TEXT	SÍ	—	Información adicional o descripción del error
ip_origen	VARCHAR(45)	SÍ	—	IP desde la que se invocó el servicio
timestamp_op	TIMESTAMP	NO	—	Marca de tiempo exacta de la operación

3.5.4. Reglas de integridad y calidad

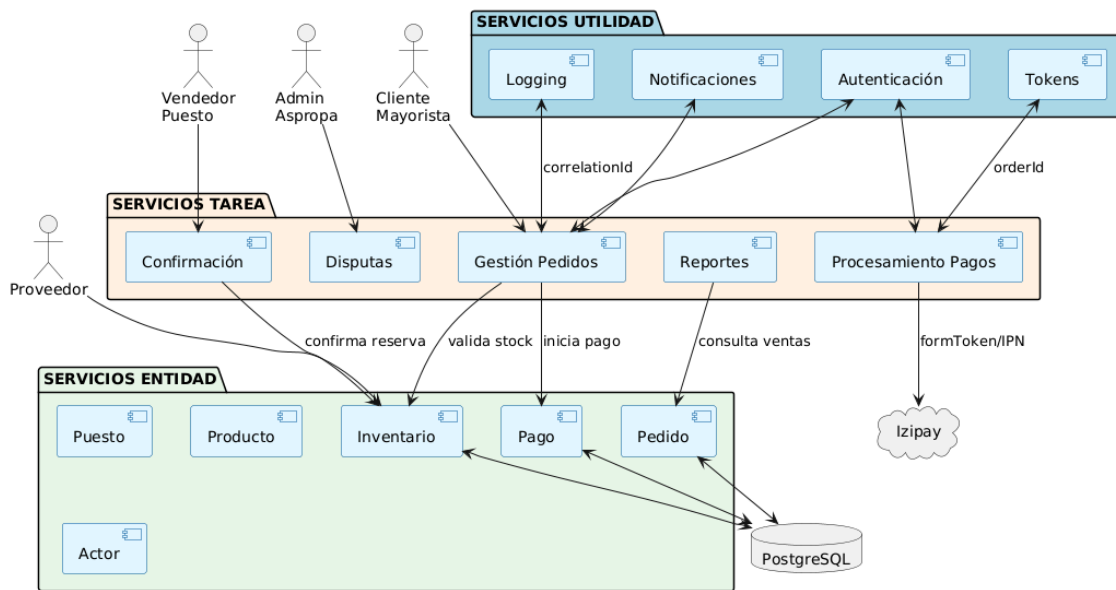
Las reglas de integridad y calidad de la base de datos de Aspropa aseguran que la información utilizada por los servicios SOA sea consistente, precisa, pertinente y trazable a lo largo de su ciclo de vida. En términos de integridad, el diseño debe preservar la unicidad de las entidades, la validez de las relaciones entre tablas y la restricción semántica de los valores admitidos en cada atributo, evitando registros huérfanos, duplicidades y estados inválidos.

Desde la perspectiva de calidad de datos, estas reglas deben alinearse con el marco peruano de protección de datos personales, en particular con la Ley N.º 29733, cuyo principio de calidad exige que los datos sean veraces, exactos, actualizados, necesarios, pertinentes y adecuados a la finalidad de su tratamiento. En consecuencia, la base de datos no solo debe responder a criterios técnicos de normalización y consistencia relacional, sino también a exigencias legales de minimización, seguridad, confidencialidad y conservación limitada de los datos personales gestionados por el sistema.

Adicionalmente, la incorporación de controles de acceso, auditoría de operaciones, validación de dominios y mecanismos de respaldo fortalece la confiabilidad del sistema y se alinea con prácticas de gestión de calidad y seguridad de la información promovidas en el contexto peruano, como las basadas en ISO 9001 e ISO/IEC 27001. De este modo, la base de datos se convierte no solo en un repositorio transaccional, sino en un componente gobernado que sostiene la interoperabilidad y la robustez del ecosistema SOA de Aspropa.

4. Tipos de servicios

La clasificación de los servicios implementados responde a la necesidad de organizar la arquitectura orientada a servicios en componentes con responsabilidades claramente diferenciadas. Para ello, se distinguen servicios de utilidad, servicios de entidad y servicios de tarea, cada uno con un propósito específico dentro de la solución propuesta para la automatización de procesos B2B en el entorno de Aspropa y el mercado popular de Ingeniería. Esta separación permite mejorar la modularidad, reducir el acoplamiento entre componentes y facilitar tanto la reutilización como el mantenimiento evolutivo del sistema.



4.1 Servicio de utilidad

Los servicios de utilidad comprenden aquellas funciones de apoyo que pueden ser utilizadas por distintos módulos del sistema sin depender directamente de una entidad de negocio específica. Su finalidad es proporcionar capacidades técnicas comunes a múltiples procesos, de manera que estas no tengan que implementarse repetidamente en cada servicio funcional. En una arquitectura SOA, este tipo de servicio contribuye a estandarizar operaciones transversales y a reforzar principios como la reutilización, la mantenibilidad y la consistencia operativa.

Dentro de esta categoría se ubican mecanismos como la autenticación de usuarios, la generación y validación de tokens, el envío de notificaciones y otras funciones técnicas compartidas. Estos servicios no representan procesos completos del negocio ni administran recursos centrales del dominio, sino que actúan como soporte reusable para que otros servicios puedan operar de forma segura, controlada y uniforme. Por ello, su valor principal radica en servir de base técnica común para distintas partes de la solución.

Servicio	Descripción	Consumidores	Operaciones principales
Autenticación y Autorización	Gestión de tokens JWT y control RBAC para todos los actores	Todos los servicios	login(), validateToken(), getPermissions(actorId)

Generación de Tokens	Creación y validación de formToken para Izipay, correlationId y orderId únicos	Servicio Pagos, Pedidos	generatePaymentToken(), generateCorrelationId()
Notificaciones	Envío de confirmaciones, alertas y avisos vía email/SMS	Todos los servicios	sendConfirmation(), sendStockAlert(), sendDisputeNotification()
Logging Centralizado	Registro estructurado con trazabilidad completa	Aspecto transversal	logEvent(correlationId, actor, operation, status)

Estos servicios tienen el propósito de eliminar duplicidad técnica y estandarizar funciones críticas como seguridad, trazabilidad e integración externa, permitiendo que los servicios de negocio se concentren exclusivamente en lógica de dominio.

4.2 Servicio de entidad

Los servicios de entidad son aquellos orientados a la gestión de los objetos principales del dominio de negocio. Su función consiste en encapsular las operaciones necesarias para registrar, consultar, actualizar y mantener la integridad de la información asociada a entidades relevantes del sistema. En este tipo de servicio se administra el ciclo de vida de los datos estructurales sobre los cuales se construyen posteriormente los procesos funcionales del sistema. En esta sección deben identificarse las entidades principales involucradas en la solución, tales como usuarios, puestos, inventarios, pedidos, proveedores, pagos o reportes, según corresponda al modelo de negocio implementado. Asimismo, resulta necesario describir qué operaciones tendrá cada una, por ejemplo creación, consulta, modificación, actualización de estado o eliminación lógica. De esta manera, los servicios de entidad no se presentan únicamente como repositorios de datos, sino como componentes responsables de preservar coherencia, integridad y acceso controlado a la información principal del sistema.

Entidad	Servicio asociado	Operaciones CRUD + específicas
Puesto	Gestión de Puestos	crearPuesto(), actualizarPuesto(), getPuesto(id), listarPuestosPorMercado()
Inventario	Gestión de Inventario	registrarProducto(), actualizarStock(id, cantidad), reservarStock(), liberarReserva(), consultarStockRealTime()
Producto	Catálogo Proveedores	publicarProducto(), actualizarPrecioB2B(), descontinuarProducto(), getProductosActivos()

Pedido	Gestión de Pedidos	crearBorrador(), actualizarEstado(), agregarItem(), calcularTotal(), validarStock()
Pago	Gestión de Pagos	registrarIntento(), actualizarEstadoIPN(), generarComprobante(), consultarEstado(orderId)
Actor	Gestión de Actores	registrarActor(), aprobarRegistro(), actualizarEstado(), getPermisos(actorId)

4.3 Servicio de tarea

Los servicios de tarea son los encargados de ejecutar procesos concretos del negocio que requieren la coordinación de varias operaciones, reglas y servicios subyacentes. A diferencia de los servicios de utilidad, que ofrecen funciones técnicas comunes, y de los servicios de entidad, que administran datos del dominio, los servicios de tarea implementan flujos funcionales completos orientados al cumplimiento de un objetivo específico dentro del sistema. En esta categoría se describen procesos como registrar pedidos, confirmar pagos, generar reportes, resolver incidencias o cualquier otra operación que implique interacción entre varias entidades y reglas de negocio. Estos servicios suelen consumir servicios de entidad y de utilidad para completar su flujo, por lo que representan la capa donde se materializa la lógica operativa del sistema. Su importancia radica en que traducen las necesidades funcionales del negocio en secuencias de acciones concretas, controladas y trazables dentro de la arquitectura SOA.

Proceso de Negocio	Servicio de Tarea	Servicios consumidos	Flujo principal
Registrar Pedido B2B	Gestión Pedidos Multi-puesto	Inventario, Actores, Pagos	1. Validar stock multi-puesto 2. Reservar provisional 3. Calcular total con descuentos 4. Pagos pendientes
Confirmar Pago Izipay	Procesamiento Pagos	Pagos, Pedidos, Notificaciones	1. Validar IPN + firma 2. Actualizar pago aprobado 3. Pedido pagado 4. Notificar vendedor

Confirmar Inventario	Confirmación Operativa	Inventario, Pedidos, Proveedores	1. Verificar stock físico 2. Confirmar reserva 3. Pedido confirmar 4. Notificar proveedor
Generar Reporte Ventas	Reportes Analíticos	Pedidos, Pagos, Inventario	1. Filtrar por período/actor 2. Consolidar métricas 3. Generar CSV/PowerBI 4. Audit trail completo
Resolver Disputa	Gestión Disputas	Todos los servicios	1. Bloquear operación 2. Revisar evidencia 3. Resolver estado 4. Notificar partes

5. SEGURIDAD DEL SERVICIO WEB

La seguridad constituye uno de los requisitos no funcionales más críticos del sistema, dado que la plataforma procesa transacciones financieras reales a través de Izipay, gestionará datos personales y comerciales de vendedores, proveedores y clientes mayoristas, y operará expuesta a internet en un entorno B2B. Ignorar este aspecto no solo comprometería la integridad de los datos, sino también la confianza de los actores y el cumplimiento normativo frente a la Ley N.º 29733 – Ley de Protección de Datos Personales del Perú. Por este motivo, la arquitectura SOA propuesta incorporará mecanismos de seguridad en todos los niveles del sistema, desde el acceso del usuario hasta la comunicación con servicios externos, siguiendo las recomendaciones del OWASP Top Ten como referencia base para identificar y mitigar las vulnerabilidades más frecuentes en aplicaciones web.

5.1 Autenticación

La autenticación es el mecanismo mediante el cual el sistema verifica que quien intenta acceder es realmente quien dice ser. Según el OWASP Top Ten, la A07:2021 – Identification and Authentication Failures identifica precisamente la ausencia o debilidad en este proceso como una de las principales causas de brechas de seguridad en aplicaciones web.

El acceso a la plataforma se realizará mediante usuario y contraseña, donde el usuario puede ser el correo electrónico o el identificador comercial registrado por el actor. La contraseña deberá cumplir con requisitos mínimos de complejidad: longitud de

al menos ocho caracteres, combinación de letras, números y un carácter especial. Las contraseñas nunca serán almacenadas en texto plano en la base de datos; en su lugar, se aplicará un algoritmo de hashing seguro como bcrypt, el cual incorpora un factor de costo y sal aleatoria, garantizando que contraseñas idénticas produzcan hashes distintos y haciendo computacionalmente inviable su reversión.

Una vez que el usuario ingresa credenciales correctas, el backend implementado en Spring Boot generará un JSON Web Token (JWT) firmado digitalmente mediante el algoritmo HMAC-SHA256. Este token encapsula el identificador del actor, su rol y la fecha de expiración, y deberá ser incluido en el encabezado Authorization: Bearer <token> en cada solicitud posterior. El servidor validará la firma del token sin necesidad de consultar la base de datos en cada petición, lo que reduce la latencia y desacopla la sesión del estado del servidor, principio coherente con la naturaleza stateless de los servicios REST. El token tendrá una vida útil limitada —por ejemplo, 60 minutos— y se podrá renovar mediante un refresh token de mayor duración, reduciendo el riesgo en caso de interceptación.

Para el caso de la integración con Izipay, la autenticación se complementa con la validación de firma HMAC en los eventos IPN recibidos, garantizando que las notificaciones de pago proviene efectivamente del proveedor externo y no de un actor malicioso. Este control mitiga directamente la A02:2021 – Cryptographic Failures del OWASP al asegurar que la comunicación con servicios externos esté verificada criptográficamente.

5.2 Autorización

La autorización determina qué operaciones puede ejecutar un actor autenticado dentro del sistema. La A01:2021 – Broken Access Control encabeza el OWASP Top Ten precisamente porque muchas aplicaciones verifican la identidad pero no restringen correctamente el alcance de las acciones permitidas. Para evitar esto, el sistema implementará un modelo de control de acceso basado en roles (RBAC – Role-Based Access Control), donde cada token JWT incluirá el rol del actor y cada endpoint validará que el rol corresponda con la operación solicitada.

El sistema contempla cuatro roles principales derivados de los actores definidos en el contrato de servicio:

Rol	Permisos principales
Vendedor de Puesto	Gestionar inventario propio, recibir y confirmar pedidos B2B entrantes, generar reportes de su puesto
Proveedor	Publicar y actualizar catálogo de productos, consultar y confirmar órdenes, actualizar precios y disponibilidad

Cliente Mayorista	Consultar productos, crear pedidos multi-puesto, realizar pagos y descargar comprobantes
Administrador	Gestionar registros de todos los actores, monitorear transacciones globales, resolver disputas, generar auditorías y configurar reglas del sistema

Cada endpoint del backend tendrá asociada una política de acceso explícita. Por ejemplo, un vendedor no puede consultar el inventario de otro puesto, un proveedor no puede modificar pedidos ajenos y un cliente mayorista no puede aprobar ni rechazar operaciones que corresponden al vendedor o al administrador. Cualquier intento de operar fuera del rol asignado generará una respuesta 403 Forbidden y quedará registrado en el log de auditoría como el evento EX-AUTH-002 ya previsto en el catálogo de excepciones. Este enfoque de least privilege —principio de mínimo privilegio— garantiza que cada actor tenga acceso únicamente a los recursos necesarios para cumplir sus responsabilidades.

5.3 Confidencialidad

La confidencialidad asegura que la información intercambiada entre los actores y el sistema no pueda ser leída o alterada por terceros durante su transmisión ni en su almacenamiento. La A02:2021 – Cryptographic Failures del OWASP señala que la exposición de datos sensibles en tránsito o en reposo sigue siendo una de las vulnerabilidades más explotadas en aplicaciones modernas.

En primer lugar, toda la comunicación entre el frontend, los servicios backend y las APIs externas como Izipay se realizará exclusivamente bajo el protocolo HTTPS con TLS 1.2 o superior, garantizando cifrado en tránsito y autenticidad del servidor mediante certificados digitales. El uso de HTTP simple será deshabilitado en entornos de producción, y cualquier redirección desde HTTP será redirigida automáticamente a HTTPS. Este control es particularmente crítico dado que el sistema procesa credenciales, datos de pago y tokens de autenticación en cada solicitud.

En cuanto a los datos almacenados, los campos sensibles en PostgreSQL, como contraseñas, datos bancarios o información de identificación de actores, serán cifrados o hasheados según su naturaleza. Las contraseñas, como se indicó, utilizarán bcrypt; los datos de pago procesados por Izipay no se almacenarán en la base de datos propia del sistema, sino que se delegan íntegramente a la pasarela, reduciendo la superficie de exposición. Las claves secretas utilizadas para firmar tokens JWT y para la comunicación con Izipay se gestionan como variables de entorno, nunca embebidas en el código fuente ni publicadas en repositorios.

Adicionalmente, el sistema deberá cumplir con la Ley N.º 29733 de Protección de Datos Personales del Perú, lo que implica registrar el tratamiento de datos personales de los actores, implementar mecanismos de consentimiento y mantener registros auditables del uso de esa información. Este cumplimiento normativo refuerza la dimensión legal de la confidencialidad, más allá de los controles técnicos.

5.4 Disponibilidad

La disponibilidad garantiza que el sistema opere de forma continua y predecible para todos los actores, incluso ante errores, picos de demanda o intentos de abuso. Los requisitos no funcionales del sistema ya establecen una disponibilidad mínima del 99% con un tiempo de inactividad máximo de 3.6 horas mensuales y soporte a 200 transacciones concurrentes en horas pico.

Para lograrlo, el sistema implementará tres líneas de control:

- Manejo de errores controlado. Todos los servicios responderán con mensajes estructurados siguiendo el estándar RFC 9457 – Problem Details, incluyendo un `correlationId` para trazabilidad. Los errores internos no expondrán trazas de pila ni detalles del sistema, evitando la A05:2021 – Security Misconfiguration del OWASP. Las excepciones previstas —desde EX-SYS-001 (error interno no controlado) hasta EX-INT-001 (servicio externo no disponible)— tienen una respuesta definida con código HTTP, mensaje seguro y estado resultante, permitiendo que el sistema degradado de forma controlada sin interrumpir flujos críticos.
- Ahora en la validación de peticiones en la capa de entrada. Toda solicitud entrante será validada antes de procesarse: campos obligatorios, tipos de datos, longitud, formato y consistencia entre entidades relacionadas como stock, pedido, precio y pago. Esta validación temprana reduce la carga sobre la base de datos y protege contra la A03:2021 – Injection del OWASP, ya que entradas malformadas o inyecciones de código serán rechazadas antes de alcanzar la lógica de negocio. El backend construido sobre Spring Boot aplicará validaciones mediante anotaciones estándar (`@NotNull`, `@Size`, `@Pattern`) complementadas con reglas de negocio específicas del dominio.
- Infraestructura estable con políticas de resiliencia. El sistema incorporará mecanismos de `retry` controlado con límite de intentos para operaciones que dependen de servicios externos, como la comunicación con Izipay. La política de `throttling` ya definida —100 consultas por hora por puesto— protegerá al sistema contra abusos y contra la A04:2021 – Insecure Design asociada a la ausencia de controles de tasa. El despliegue previsto en Heroku permitirá escalar horizontalmente según la demanda, y la combinación con Git/GitHub como control de versiones garantizará que los despliegues sean reproducibles y auditables. Adicionalmente, los logs estructurados con `timestamp`, `usuario`, `correlationId` y `servicio origen` permitirán detectar anomalías de disponibilidad antes de que impacten a los actores del sistema.

PROYECTO: ARQUITECTURA ORIENTADA AL SERVICIO

AVANCE N.º 2 – AVANCE EN EL DESARROLLO DE SERVICIO WEB

1. Arquitectura de software en tiempos de ejecución

Una arquitectura rígida en SOA significa que las conexiones entre servicios están fijadas en el código. En tiempo de ejecución, el sistema es incapaz de modificarse. Si un componente falla o cambia, todo el sistema se rompe y requiere compilar y desplegar el código de nuevo. Una arquitectura flexible en SOA significa que el sistema se modifica mediante configuraciones externas sin alterar el código fuente. En tiempo de ejecución, permite cambiar rutas o parámetros editando archivos o bases de datos. Sin embargo, requiere que un humano aplique el cambio o reinicie el servicio. Una arquitectura adaptativa en SOA significa que el sistema se configura de forma autónoma en tiempo real. En tiempo de ejecución, toma decisiones automáticas según el entorno mediante reglas y registros dinámicos. El sistema detecta fallas, busca reemplazos y desvía el tráfico por sí mismo sin intervención humana.

1.1. Arquitectura rígida

1.2. Arquitectura flexible

1.3. Arquitectura adaptativa

2. Modelos de distribución y Licenciamiento

Se aplicará en lo que es para poder aplicarse mediante un tratado y la aplicación del sistema mediante un plan freemium debido al contexto precario económicamente y los recursos adaptativos. Las licencias son claves para poder rendir frutos en un trabajo, la forma de ejecutar va a ser mediante forma mensual en suscripciones.

3. Diagramas técnicos

Aplicación de los diagramas para poder ejecutarlo de forma ideal

3.1. Diagramas de interacción

3.2. Diagramas de despliegue

3.3. Diagramas de componentes

3.4. Diagramas CRC

4. Desarrollo del servicio web

4.1. Evidencia de código y estructura

4.2. Pruebas de funcionamiento

4.3. Descripción del avance

6. CONCLUSIONES

Este primer avance ha permitido sentar las bases sólidas del sistema MercadoLink, una plataforma de servicios web basada en Arquitectura Orientada a Servicios (SOA) diseñada para modernizar y digitalizar la operación de la Asociación de Comerciantes del Mercado Popular de Ingeniería (SMP) en Lima.

A lo largo de esta etapa se identificaron con claridad los principales problemas que enfrenta el mercado: la fuerte dependencia del efectivo, que genera ineficiencias operativas; la falta de control de inventarios en tiempo real; y las dificultades para cumplir con las obligaciones tributarias. Frente a esta realidad, se propuso una solución

técnica bien fundamentada que aborda cada una de estas necesidades mediante servicios especializados, modulares e interoperables.

La elección de SOA como enfoque arquitectónico resulta especialmente adecuada para este contexto. Al descomponer el sistema en servicios de utilidad, entidad y tarea, cada componente puede evolucionar de manera independiente, lo que facilita su mantenimiento y garantiza una excelente escalabilidad a futuro.

La pila tecnológica seleccionada —Java con Spring Boot, Apache Kafka, PostgreSQL, Docker y Kubernetes— se alinea perfectamente con los requerimientos del proyecto: alto rendimiento, tolerancia a fallos e integración nativa con las principales soluciones de pago digital en Perú (Yape y Plin). Asimismo, los mecanismos de seguridad contemplados (JWT, RBAC, HTTPS/TLS 1.3 y cifrado de datos en reposo) aseguran que la información financiera se maneje con los estándares de protección necesarios.

7. BIBLIOGRAFÍA

- Adelgado, V. (s. f.). *Introducción a SOA: Elementos estructurales de un servicio*. Facultad de Ingeniería, Universidad de la República – FING.
<https://www.fing.edu.uy/~adelgado/ExtensionSOA/material/IntroduccionSOA.pdf>
- Apache Software Foundation. (2023). *Apache Tomcat 9.0 – Security considerations*.
<http://tomcat.apache.org/>
- Aranda, G., & Vieira, M. (2010). Integración de procesos de negocio aplicando la arquitectura orientada a los servicios (SOA). *Dialnet*.
<https://dialnet.unirioja.es/descarga/articulo/6230448.pdf>
- BPM Institute. (2014). *Service Oriented Architecture – A service contract design template*.
<https://www.bpminstitute.org/resources/articles/service-oriented-architecture-service-contract-design-template/>
- CompTIA. (2022). *Security+ exam objectives: Authentication, authorization, and accounting*. <https://www.comptia.org/>
- Daswani, N., Kern, C., & Kesavan, A. (2007). *Foundations of security: What every programmer needs to know*. Apress.
- De Ryck, P., Maex, M., & Desmet, L. (2019). *Web security academy: CSRF protection*. PortSwigger. <https://portswigger.net/web-security>
- DZone. (2019). *Service oriented architecture patterns – Reference card*.
<https://dzone.com/refcardz/soa-patterns>
- Erl, T. (2009). *Service-oriented architecture: Analysis and design for enterprise integration*. Prentice Hall Professional.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].
- Firebase by Google. (2023). *Firebase documentation and guides*.
<https://firebase.google.com/docs>
- GitHub, Inc. (2023). *GitHub guides: Hello world*. <https://guides.github.com/>

Harris, S. (2018). *CISSP all-in-one exam guide* (7^a ed.). McGraw-Hill Education.

Heroku. (2023). *Heroku platform as a service (PaaS) – Documentation*.
<https://www.heroku.com/>

IEEE. (2011). *ISO/IEC/IEEE 42010:2011 – Systems and software engineering – Architecture description*. Institute of Electrical and Electronics Engineers.

Internet Engineering Task Force. (2015). *RFC 7519: JSON Web Token (JWT)*.
<https://tools.ietf.org/html/rfc7519>

Internet Engineering Task Force. (2023). *RFC 9457: Problem details for HTTP APIs*.
<https://tools.ietf.org/html/rfc9457>

Josuttis, N. M. (2007). *SOA in practice: The art of distributed system design*. O'Reilly Media.

McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley Professional.

Microsoft Corporation. (2023). *Power BI – Documentation and tutorials*.
<https://docs.microsoft.com/power-bi/>

MITRE Corporation. (2022). *2021 CWE top 25 most dangerous software weaknesses*.
<https://cwe.mitre.org/top25/>

Mohor Betancur, C. R. (2006). *Service oriented architecture (SOA): Concepts, technology, and design*. Pontificia Universidad Católica de Valparaíso.
http://opac.pucv.cl/pucv_txt/txt-7500/uci7565_01.pdf

National Institute of Standards and Technology. (2019). *Cybersecurity framework version 1.1*. <https://www.nist.gov/cyberframework>

Object Management Group. (2012). *Enterprise SOA with SoAML*. OMG White Paper.
<https://www.omg.org/news/whitepapers/EnterpriseSoAML.pdf>

OASIS. (2006). *Web services security (WS-Security) version 1.1*. OASIS Standard.
<http://docs.oasis-open.org/wss/v1.1/>

OpenAPI Initiative. (2023). *OpenAPI specification 3.1.0*.
<https://spec.openapis.org/oas/v3.1.0>

OWASP. (2021). *OWASP Top 10 – 2021*. Open Worldwide Application Security Project.
<https://owasp.org/Top10/>

OWASP. (2021). *A01:2021 – Broken access control*. En *OWASP Top 10 – 2021*.
https://owasp.org/Top10/A01_2021-Broken_Access_Control/

OWASP. (2021). *A02:2021 – Cryptographic failures*. En *OWASP Top 10 – 2021*.
https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

OWASP. (2021). *A03:2021 – Injection*. En *OWASP Top 10 – 2021*.
https://owasp.org/Top10/A03_2021-Injection/

OWASP. (2021). *A04:2021 – Insecure diseño*. En *OWASP Top 10 – 2021*.
https://owasp.org/Top10/A04_2021-Insecure_Design/

OWASP. (2021). *A05:2021 – Security misconfiguration*. En *OWASP Top 10 – 2021*.
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

OWASP. (2021). *A07:2021 – Identification and authentication failures*. En *OWASP Top 10 – 2021*. https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Perl, E. A. (2009). *Introduction to service oriented architectures*. Springer-Verlag.

Perú, República del. (2011). *Ley N.º 29733 – Ley de protección de datos personales*. Diario Oficial El Peruano.

Perú, República del. (2013). *Decreto Supremo N.º 003-2013-JUS – Reglamento de la Ley de Protección de Datos Personales*. Diario Oficial El Peruano.

Pivotal Software & VMware. (2023). *Spring Boot reference documentation*. <https://spring.io/projects/spring-boot>

Plantilla de contrato de integración para web services JOC-ER. (s. f.). Generalitat Valenciana – Hacienda Pública. https://hisenda.gva.es/documents/166579797/166579894/CONTRATO_INTEGRACION_JOC-ER-GuiaDesasistida_v01.02.pdf

PostgreSQL Global Development Group. (2023). *PostgreSQL 15 documentation*. <https://www.postgresql.org/>

Salter, C., & Xu, D. (2006). A framework for fast, flexible, and comprehensive security audits. En *Proceedings of the 22nd Annual Computer Security Applications Conference* (pp. 189-198). IEEE Computer Society.

SANS Institute. (2019). *SANS top 25 software errors*. <https://www.sans.org/top25-software-errors/>

Springer, J. (2019). *Spring security in action*. Manning Publications.

Startup Legal. (2023). *Plantilla de contrato de desarrollo de software para startups*. <https://startuplegal.es/>

The PostgreSQL Global Development Group. (2023). *PostgreSQL 15 documentation*. <https://www.postgresql.org/> (Nota: repetido, se mantiene una sola entrada)

Universidad de la República – Facultad de Ingeniería (FING). (2006). *Introducción a SOA: Componentes, estructura y especificación de servicios*. Instituto de Computación (INCO). <https://www.fing.edu.uy/>

Universidad Piloto de Colombia. (2018). *Marco de referencia para el diseño de servicios web en arquitecturas SOA*. Repositorio Institucional. <https://repository.unipiloto.edu.co/Modelo-3-Contrato-para-el-desarrollo-de-Software>. (2021). Oficina de Transferencia de Resultados de Investigación (OTRI), Universidad Miguel Hernández. <https://otri.umh.es/files/2021/03/Modelo-3-Contrato-para-el-desarrollo-de-Software.docx>

Modelo de contrato para desarrollo de un sistema de software con requisitos cerrados. (2011). Scribd. <https://es.scribd.com/document/52629501/MODELO-DE-CONTRATO-PARA-DESARROLLO-DE-UN-SISTEMA-DE-SOFTWARE-CON-REQUISITOS-CERRADOS>

World Wide Web Consortium. (2004). *Web services description language (WSDL) 1.1*. <https://www.w3.org/TR/wsdl>

World Wide Web Consortium. (2011). *SOAP version 1.2 part 1: Messaging framework*.
<https://www.w3.org/TR/soap12-part1/>